

STUDY GUIDE · FROM "NEVER INSTALLED KUBERNETES" TO FOUR  
WORKING CLUSTERS

# Kubernetes From Zero Installation & Cluster Setup, Hands-On

13 lessons for complete beginners: what Kubernetes is, the architecture, Linux prerequisites, container runtimes and the CRI, kubeadm, CNI networking, Minikube, MicroK8s, Amazon EKS, cluster verification, cleanup and choosing an installation method; plus the troubleshooting method, the capstone, a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-kubernetes-from-zero>

# Contents

---

## Study material

- 01 · What is Kubernetes?
- 02 · Kubernetes architecture
- 03 · Installation prerequisites (Linux)
- 04 · The container runtime and the CRI
- 05 · What kubeadm does, step by step
- 06 · CNI and Pod networking
- 07 · The four installation methods
- 08 · How Minikube works inside
- 09 · How MicroK8s works inside
- 10 · Amazon EKS architecture
- 11 · The cluster verification checklist
- 12 · Cleanup and reset
- 13 · Comparing the installation methods

## Troubleshooting labs

## Capstone · prove it

## Glossary

## Interview questions

Each lesson explains what a part of the installation is, why it exists, how it works and where you meet it at work, and ends with "check yourself" questions. The hands-on installation lessons (kubeadm, Minikube, MicroK8s, EKS), the eight troubleshooting labs with real outputs, the challenges and the guided tutorial are in the repository.

# Study material

---

Everything you need to revise after (or alongside) the hands-on lessons.

| File                                | What it is                                                                                                                                                       | Use it when                                  |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|
| <code>study-guide.pdf</code>        | the whole course as one printable PDF: the docs, the guided chapters and the answers, with a table of contents (built by <code>study/tools/build_pdf.py</code> ) | you want to read offline, print, or annotate |
| <code>glossary.md</code>            | every term used in the course, A–Z, in one or two plain sentences, with a link to where it is explained                                                          | a word in a lesson is unclear                |
| <code>interview-questions.md</code> | 25 questions with model answers, from fundamentals to EKS and operations                                                                                         | before an interview, or to test yourself     |
| <code>../tutorial/</code>           | the guided training course through the repository                                                                                                                | you want to be taught, step by step          |

## How to revise

1. Work through the [tutorial](#); it sends you to the lessons, labs and docs in the right order.
2. After each chapter, look up every glossary term you could not explain to a colleague.
3. At the end, answer the interview questions **out loud**, then compare with the model answers. Explaining is a different skill from knowing; it needs practice.
4. Finish with the [capstone](#) and the [knowledge check](#).

# 01 · What is Kubernetes?

Time: 15 minutes · No commands in this chapter: it explains what you are about to install, and why.

## The problem it solves

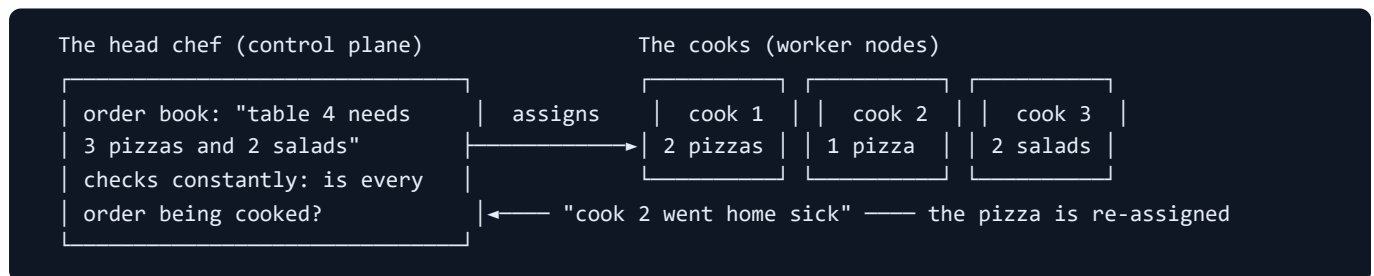
With Docker you can run an application in a container on **one** machine. Real applications quickly outgrow that:

- one machine is not enough, so containers must be spread over **many** machines
- a machine dies at 3 a.m., and its containers must start somewhere else, without a human
- the shop has 10 times more visitors on Black Friday, so it needs 10 more copies of the web server, then fewer again
- a new version must go live without downtime, and roll back if it is broken
- containers move between machines, so their IP addresses change all the time: how do they find each other?

Doing this by hand, or with scripts, does not scale. **Kubernetes** is the system that does it for you. It is an open-source **container orchestrator**: you give it a group of machines and a description of what should run, and it keeps reality matching that description.

## An analogy

Think of a large restaurant kitchen.



The head chef does not cook. They keep the order book (the **desired state**), hand the work to cooks with free hands (**scheduling**), and keep checking that every order is being cooked. If a cook leaves, the chef gives the work to another one (**self-healing**). You talk only to the chef.

## What Kubernetes does

| Feature               | What it means                                                                   | You will see it in                           |
|-----------------------|---------------------------------------------------------------------------------|----------------------------------------------|
| <b>Desired state</b>  | You declare <i>what</i> you want ("2 copies of nginx"), not <i>how</i> to do it | every <code>kubectl create deployment</code> |
| <b>Reconciliation</b> | Controllers compare desired and actual state in a loop, and fix the difference  | the replacement Pods after a node fails      |
| <b>Scheduling</b>     | The scheduler picks a node with enough free CPU and memory for each Pod         | <a href="#">troubleshooting 08</a>           |

| Feature                  | What it means                                                                           | You will see it in                           |
|--------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------|
| <b>Self-healing</b>      | Crashed containers are restarted, Pods on dead nodes are replaced                       | <a href="#">troubleshooting 01</a>           |
| <b>Service discovery</b> | A <b>Service</b> gives a group of Pods one stable name and IP; CoreDNS answers the name | the <code>web</code> Service in every lesson |
| <b>Rolling updates</b>   | A new version replaces Pods a few at a time; a rollback is one command                  | <code>kubectl rollout status</code>          |
| <b>Scaling</b>           | Change the number of copies with one command, or automatically                          | <code>kubectl scale</code>                   |

The basic objects you will meet in the lessons:

```
Deployment "web" — "I want 2 copies of nginx:1.30-alpine"
├─ ReplicaSet — keeps exactly 2 Pods alive
│   ├─ Pod web-...-abcde (one or more containers, one IP) on node A
│   └─ Pod web-...-fghij on node B
Service "web" — one stable address in front of all Pods with the label app=web
```

## What Kubernetes is not

- **Not a container runtime.** It does not run containers itself; it asks a runtime such as containerd to do it ([docs/04](#)).
- **Not a way to build images.** You build images with Docker (or another builder) and push them to a registry.
- **Not a full platform out of the box.** Ingress controllers, storage drivers, monitoring, logging, CI/CD and backups are added on top. Clusters differ mostly in these extras.
- **Not always the right answer.** One small application on one server is simpler with Docker Compose. Kubernetes pays off when you have several services, several machines, or need high availability.

## Where Kubernetes runs

| Kind                | Who runs the control plane       | Examples                                      | Used for                                      |
|---------------------|----------------------------------|-----------------------------------------------|-----------------------------------------------|
| <b>Local</b>        | you, on your computer            | Minikube, MicroK8s, kind                      | learning, development, testing manifests      |
| <b>Self-managed</b> | you, on your own machines or VMs | kubeadm, MicroK8s clusters, Rancher RKE2, k3s | on-premises, data centres, edge, full control |
| <b>Managed</b>      | the cloud provider               | Amazon EKS, Google GKE, Azure AKS             | most production workloads in the cloud        |

This course installs one of each: [kubeadm](#) (self-managed, the "hard way made official"), [Minikube](#) and [MicroK8s](#) (local and lightweight), and [Amazon EKS](#) (managed). The current Kubernetes version used throughout is **v1.37**.

## Real-world context

Most companies that run containers in production run them on Kubernetes, very often a managed service. As a DevOps engineer you will be expected to: create clusters (with infrastructure as code), keep them updated, deploy applications to them, and above all **troubleshoot** them when something goes wrong. Understanding what happens during installation is the best preparation for troubleshooting: every component you install by hand here is one you will one day debug.

## Check yourself



### 1. What does "desired state" mean?

You describe what you want to exist (for example 2 copies of nginx), and Kubernetes works continuously to make the actual state match it. You do not write the steps.



### 1. A node with 3 of your Pods is switched off. What happens?

The node becomes NotReady. After a timeout, the controllers notice the Pods are gone and the Deployment's ReplicaSet creates replacement Pods, which the scheduler places on healthy nodes.



### 1. Why do applications use a Service instead of Pod IP addresses?

Pods are replaced all the time and every new Pod gets a new IP. A Service has a stable IP and DNS name and sends traffic to whichever Pods currently match its label selector.



### 1. Does Kubernetes start the containers itself?

No. The kubelet on each node asks the container runtime (containerd in this course) through the CRI to start them.



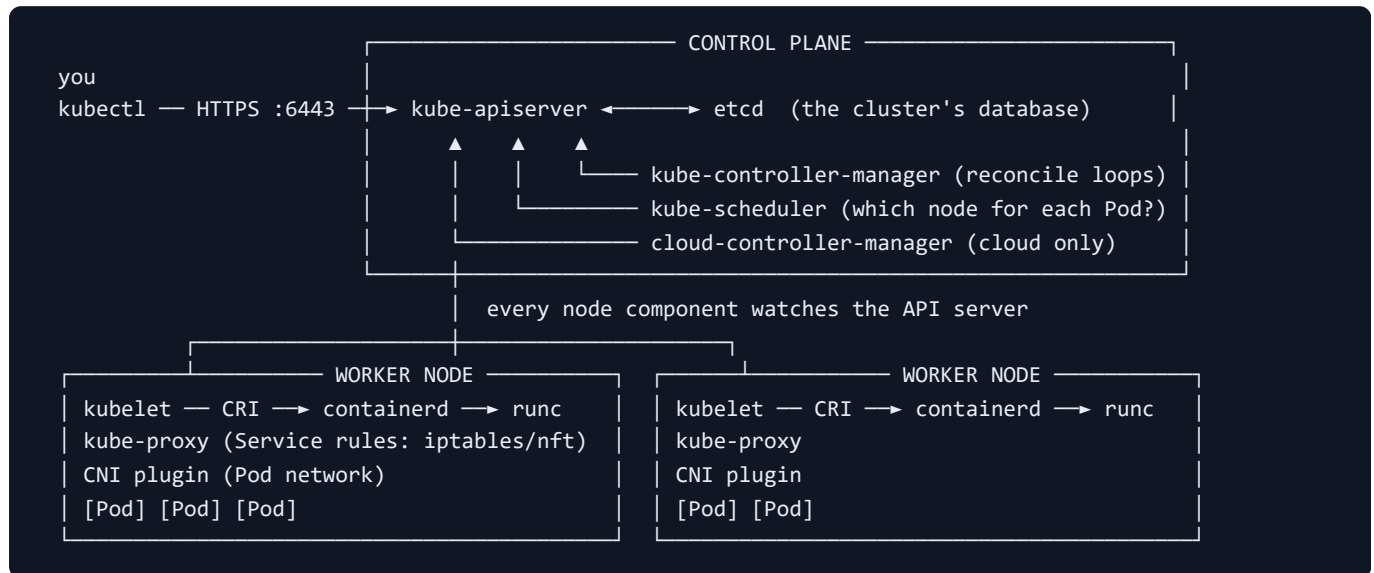
### 1. When is Kubernetes probably overkill?

For a single small application on one server, where Docker Compose is simpler to run and to understand.

## 02 · Kubernetes architecture

Time: 25 minutes · Every installation lesson refers back to this page: it is the map of what you install.

A Kubernetes cluster is two kinds of machines: the **control plane**, which decides, and the **worker nodes**, which run your containers. Everything talks through one door: the **API server**.



### Control plane components

| Component                       | Job                                                                                                                                                                                                           | If it is down                                                                           |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>kube-apiserver</b>           | The front door. Every client (kubectl, kubelets, controllers) reads and writes cluster state only through it. Authenticates, authorises, validates, then stores objects in etcd. Listens on TCP <b>6443</b> . | Nothing can be changed or read; running Pods keep running                               |
| <b>etcd</b>                     | A consistent, distributed key-value store: the <b>only</b> place the cluster state lives. Ports 2379–2380.                                                                                                    | The API server cannot work. Losing etcd's data = losing the cluster's state: back it up |
| <b>kube-scheduler</b>           | Watches for Pods with no node, picks a node that fits (free CPU/memory requests, taints, affinity), writes the decision back                                                                                  | New Pods stay <b>Pending</b>                                                            |
| <b>kube-controller-manager</b>  | Runs the control loops: Deployment, ReplicaSet, Node (marks nodes NotReady, evicts), Job, ServiceAccount, endpoints...                                                                                        | Nothing reacts: no replacement Pods, no rollouts                                        |
| <b>cloud-controller-manager</b> | Only in clouds: talks to the provider's API, for example to create a load balancer for a <code>type: LoadBalancer</code> Service                                                                              | Cloud load balancers and node metadata are not managed                                  |

### Node components

| Component                | Job                                                                                                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kubelet</b>           | The agent on every node. Registers the node, watches the API server for Pods assigned to it, asks the runtime to start them, reports their status and the node's health ( Ready ) |
| <b>container runtime</b> | Pulls images and runs containers. The kubelet talks to it through the <b>CRI</b> (Container Runtime Interface). This course uses <b>containerd 2.x</b> (docs/04)                  |
| <b>kube-proxy</b>        | Turns Services into packet-forwarding rules (iptables or nftables) on every node, so a Service IP reaches a Pod                                                                   |

## Add-ons every cluster needs

| Add-on                                                            | Job                                                                                             |
|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>CNI plugin</b> (Flannel, Calico, Cilium, AWS VPC CNI...)       | Gives every Pod an IP and connects Pods across nodes. Without it, nodes stay NotReady (docs/06) |
| <b>CoreDNS</b>                                                    | Cluster DNS: answers names like <code>web.default.svc.cluster.local</code> with Service IPs     |
| optional: metrics-server, an Ingress controller, a storage driver | <code>kubectl top</code> , HTTP routing, persistent volumes                                     |

## What happens when you run `kubectl apply`

Follow one Deployment of 2 nginx Pods through the whole system:

1. `kubectl` reads `~/.kube/config`, sends the Deployment to the API server over HTTPS
2. API server: authenticate (certificate/token) → authorise (RBAC) → admission → validate → store in etcd
3. Deployment controller (controller-manager) sees a new Deployment → creates a ReplicaSet
4. ReplicaSet controller sees "want 2, have 0" → creates 2 Pod objects (no node yet)
5. Scheduler sees 2 Pods without a node → scores the nodes → writes nodeName into each Pod
6. The kubelet on each chosen node sees "a Pod for me" → asks containerd (CRI) to pull the image and start it
7. The CNI plugin gives the Pod its IP; the kubelet reports the Pod Running
8. Endpoints of the Service are updated; kube-proxy on every node updates its rules; CoreDNS answers the name

Notice: no component tells another what to do directly. Each one **watches** the API server and reacts to changes. That is why a broken component shows up as "something stops happening" rather than an error message somewhere else.

## Where the components live, per installation method

|                                           | kubeadm (lesson)                                                                      | Minikube (lesson)                                                          | MicroK8s (lesson)                                                   | Amazon EKS (lesson)                           |
|-------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------|
| API server, scheduler, controller manager | <b>static Pods</b> started by the kubelet from <code>/etc/kubernetes/manifests</code> | static Pods inside one <b>container</b> (the "node") on your Docker engine | one process, <b>kubelite</b> , run by the snap as a systemd service | run by <b>AWS</b> , on machines you never see |

|                             | kubeadm (lesson)                        | Minikube (lesson)                             | MicroK8s (lesson)                          | Amazon EKS (lesson)                                      |
|-----------------------------|-----------------------------------------|-----------------------------------------------|--------------------------------------------|----------------------------------------------------------|
| Cluster datastore           | etcd (static Pod)                       | etcd inside the node container                | <b>dqlite</b> (distributed SQLite)         | etcd run by AWS                                          |
| kubelet + runtime           | systemd service + containerd on each VM | inside the node container                     | inside kubelite + containerd from the snap | on your EC2 worker nodes (Amazon Linux 2023, containerd) |
| CNI                         | you install it (Flannel)                | built in                                      | Calico, preinstalled                       | Amazon VPC CNI ( <code>aws-node</code> Pods)             |
| Who fixes the control plane | you                                     | you ( <code>minikube delete</code> and start) | you                                        | AWS                                                      |

## Check yourself



### 1. Which component is the only one that talks to etcd?

The kube-apiserver. Every other component reads and writes state through the API server.



### 1. A new Pod stays Pending and its events say nothing was scheduled. Which component made that decision?

The kube-scheduler: it found no node that fits (for example `Insufficient cpu`), see troubleshooting 08.



### 1. What does the kubelet do, and what does it not do?

It registers the node, watches for Pods assigned to it, has the container runtime start them and reports status. It does not decide where Pods run, and it does not run containers itself (the runtime does).



### 1. Why are there no kube-apiserver Pods on MicroK8s or EKS?

On MicroK8s the control plane runs inside the kubelite service; on EKS AWS runs it outside your account. Only kubeadm and Minikube run the control plane as (static) Pods you can see.



### 1. Which add-on must be installed before a kubeadm node becomes Ready?

A CNI plugin. Without a Pod network the kubelet reports the node NotReady.

## 03 · Installation prerequisites (Linux)

Time: 20 minutes · Read before the [kubeadm lesson](#). Minikube, MicroK8s and EKS do these steps for you, but they still happen underneath.

A Linux machine does not become a Kubernetes node just by installing packages. The kernel, the network settings and the resources must match what Kubernetes expects. Each step below says **what** to do and **why**. The repository automates all of them in [kubeadm/scripts/prepare-node.sh](#).

### Checklist

| # | Requirement                                                                                     | Why                                                                                                                                                                                | Check                                                                                               |
|---|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1 | <b>2 CPUs, about 2 GB RAM</b> or more on a control plane; a few GB of free disk                 | kubeadm's preflight refuses fewer than 2 CPUs or less than 1700 MB of memory: the control plane alone needs that much                                                              | <code>nproc</code> , <code>free -h</code> , <code>df -h /</code>                                    |
| 2 | <b>Unique hostname, MAC address and product_uuid</b>                                            | Nodes are identified by hostname; some components rely on unique MACs and the machine UUID. Cloned VMs often share them                                                            | <code>hostname</code> , <code>ip link</code> , <code>sudo cat /sys/class/dmi/id/product_uuid</code> |
| 3 | <b>Swap off</b>                                                                                 | By default the kubelet refuses to start with swap on: memory requests and limits assume real memory                                                                                | <code>swapon --show</code> prints nothing                                                           |
| 4 | <b>Kernel modules <code>overlay</code> and <code>br_netfilter</code></b>                        | <code>overlay</code> : the filesystem containerd uses for image layers.<br><code>br_netfilter</code> : lets iptables see traffic crossing Linux bridges, which Pod networking uses | <code>lsmod   grep -E 'overlay br_netfilter'</code>                                                 |
| 5 | <b><code>sysctl net.ipv4.ip_forward = 1</code></b>                                              | A node must route packets between Pods, other nodes and the outside. Off = Pods on different nodes cannot talk ( <a href="#">troubleshooting 02</a> )                              | <code>sysctl net.ipv4.ip_forward</code>                                                             |
| 6 | <b><code>sysctl net.bridge.bridge-nf-call-iptables = 1</code> (and <code>ip6tables</code> )</b> | Service rules (kube-proxy) are iptables rules; bridged Pod traffic must pass through them                                                                                          | <code>sysctl net.bridge.bridge-nf-call-iptables</code>                                              |
| 7 | <b>Time synchronised</b>                                                                        | Certificates and tokens have validity periods; clocks that drift make TLS fail in confusing ways                                                                                   | <code>timedatectl</code> shows <code>System clock synchronized: yes</code>                          |
| 8 | <b>Required ports open between nodes</b>                                                        | See the table below                                                                                                                                                                | <code>nc -zv &lt;ip&gt; 6443</code> from another node                                               |
| 9 | <b>A CRI container runtime, systemd cgroup driver</b>                                           | The kubelet starts containers only through the CRI; kubelet and runtime must use the same cgroup driver                                                                            | <a href="#">docs/04</a>                                                                             |

Make module and sysctl settings permanent, or they are lost at the next reboot:

```
/etc/modules-load.d/k8s.conf  overlay
                               br_netfilter
/etc/sysctl.d/k8s.conf         net.ipv4.ip_forward = 1
                               net.bridge.bridge-nf-call-iptables = 1
                               net.bridge.bridge-nf-call-ip6tables = 1
/etc/fstab                     swap line commented out
```

## About swap

Recent Kubernetes versions *can* run with swap (the kubelet's `NodeSwap` support, configured with `failSwapOn: false` and a swap behaviour). It is an opt-in, advanced setup. For a first cluster, and for most production clusters today, swap stays off.

## Ports

| Node          | Port (TCP)  | Used by                  | Who connects                                                 |
|---------------|-------------|--------------------------|--------------------------------------------------------------|
| control plane | <b>6443</b> | kube-apiserver           | everyone: kubectl, kubelets, Pods                            |
| control plane | 2379–2380   | etcd client and peer API | API server, other etcd members                               |
| control plane | 10250       | kubelet API              | API server ( <code>kubectl logs</code> , <code>exec</code> ) |
| control plane | 10257       | kube-controller-manager  | itself (health)                                              |
| control plane | 10259       | kube-scheduler           | itself (health)                                              |
| worker        | 10250       | kubelet API              | API server                                                   |
| worker        | 30000–32767 | NodePort Services        | clients of your applications                                 |

Your CNI adds its own: Flannel's VXLAN uses **UDP 8472** between all nodes. In a cloud these ports are opened with **security groups**; on-premises with the firewall ( `ufw` , `firewalld` , `nftables` ) or network ACLs. A blocked 6443 makes a worker's `kubeadm join` hang ([troubleshooting 04](#)).

## cgroups and the cgroup driver

Linux **control groups (cgroups)** limit and account the CPU and memory of a group of processes: this is how a container's `resources.limits` are enforced. Modern distributions (Ubuntu 24.04 included) use **cgroup v2** with **systemd** as the manager of the cgroup tree.

The kubelet and the container runtime must use the **same cgroup driver**. Since Kubernetes 1.22 `kubeadm` configures the kubelet with the `systemd` driver, so `containerd` must be set to `systemd` too ( `SystemdCgroup = true` ). Two managers of the same tree = instability under resource pressure.

```
systemd (manages the cgroup tree)
├─ kubelet.service           cgroupDriver: systemd
└─ kubepods.slice
   └─ kubepods-burstable-pod<uid>.slice
      └─ container (runc, via containerd: SystemdCgroup = true)
```

## Real-world context

In production these steps live in a machine image (Packer), cloud-init, or configuration management (Ansible), never typed by hand. Managed node images such as the EKS-optimised Amazon Linux 2023 AMI ship with all of them done.

## Check yourself



### 1. Why must `net.ipv4.ip_forward` be 1?

The node must route packets from Pods to other nodes and outside. With forwarding off, cross-node Pod traffic is dropped (troubleshooting 02 breaks it on purpose).



### 1. What does `br_netfilter` do?

It makes traffic crossing a Linux bridge visible to iptables, so the Service rules that kube-proxy writes also apply to Pod traffic on the bridge.



### 1. Which port must a worker reach to join?

TCP 6443 on the control plane (the API server).



### 1. Two VMs were cloned from the same image. What can go wrong?

They can share the hostname, MAC address or product\_uuid, which breaks node identity and networking. Check and fix these before installing Kubernetes.



### 1. What happens if the kubelet uses the systemd cgroup driver and containerd uses cgroupfs?

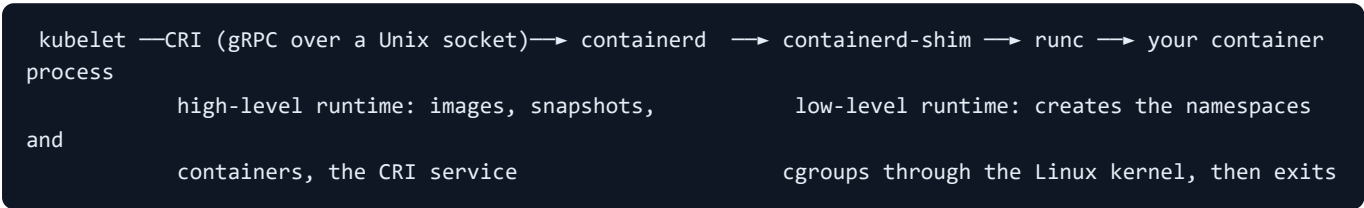
Two different managers handle the same cgroups; the node becomes unstable under pressure. Set `SystemdCgroup = true` in containerd so both use systemd.

# 04 · The container runtime and the CRI

Time: 20 minutes · Background for Step 3 of the [kubeadm lesson](#) and for troubleshooting [03](#) and [07](#).

## What a container runtime is

A **container runtime** is the software that actually runs containers on a machine: it pulls images, unpacks their layers, sets up namespaces and cgroups, and starts the process. Two layers are involved:



- **High-level runtime** (containerd, CRI-O): manages images and container lifecycles, and serves the CRI.
- **Low-level runtime** (runc, crun): the small program that really creates the container, following the OCI spec.

## The CRI

The **Container Runtime Interface** is a gRPC API that the kubelet uses to talk to *any* runtime: "run this Pod sandbox", "pull this image", "start this container", "list containers". Any runtime that implements it works with Kubernetes. For containerd the socket is `unix:///run/containerd/containerd.sock`.

## Why Docker is not used directly (dockershim)

Docker Engine predates the CRI and never implemented it. For years the kubelet contained an adapter, **dockershim**, to translate. It was removed in **Kubernetes 1.24** (2022). What that means in practice:

- Kubernetes talks to containerd (or CRI-O) directly. Docker itself also uses containerd underneath.
- **Images built with Docker work unchanged:** they are standard OCI images.
- On a node, `docker ps` shows nothing of Kubernetes. Use `crictl`.

## containerd vs CRI-O

|                   | containerd                                                            | CRI-O                                               |
|-------------------|-----------------------------------------------------------------------|-----------------------------------------------------|
| Origin            | split out of Docker, CNCF graduated                                   | built only for Kubernetes (Red Hat), CNCF graduated |
| Scope             | general-purpose (Docker, BuildKit, nerdctl also use it)               | only what the CRI needs                             |
| Where you meet it | most distributions, EKS, GKE, AKS, Minikube, MicroK8s, kubeadm guides | OpenShift, Fedora/RHEL-based clusters               |

Both are fine choices. This course uses **containerd 2.x** everywhere.

## containerd 2.x configuration

Ubuntu 24.04's own `containerd` package is the 1.x series. Kubernetes 1.37 is the last release line that supports containerd 1.x, so the lessons install `containerd.io` (2.x) from Docker's apt repository.

Two configuration traps, both in `/etc/containerd/config.toml` :

- 1. **The `containerd.io` package ships a config made for Docker**, with `disabled_plugins = ["cri"]`. `containerd` runs, but the CRI service is off and `kubeadm`'s preflight fails. Fix: generate the full default config with `containerd config default`. Reproduced on purpose in [troubleshooting 03](#).
- 2. **The `cgroup driver`**. `runc` must use `systemd` cgroups, matching the `kubelet`. In `containerd 2.x` (config version 3) the setting lives here:

```
[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]
  SystemdCgroup = true
```

(In `containerd 1.x` the path was `[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.runc.options]`; old blog posts still show it. In 2.x that section no longer applies.)

The lesson's three lines:

```
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl restart containerd
```

## crictl vs docker vs nerdctl

| Tool                 | Talks to                                                | Use it for                                                                                                                                                              |
|----------------------|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>crictl</code>  | any CRI runtime, through the CRI socket                 | <b>debugging Kubernetes nodes:</b> <code>crictl ps</code> , <code>crictl pods</code> , <code>crictl logs</code> , <code>crictl info</code> , <code>crictl images</code> |
| <code>docker</code>  | Docker Engine                                           | building and running containers on your computer; not present on a <code>kubeadm</code> node                                                                            |
| <code>nerdctl</code> | <code>containerd</code> directly (Docker-like commands) | working with <code>containerd</code> without Docker; sees Kubernetes containers in the <code>k8s.io</code> namespace                                                    |
| <code>ctr</code>     | <code>containerd</code> directly (low level)            | <code>containerd</code> 's own debug client; not user friendly                                                                                                          |

`crictl` is installed with the Kubernetes packages ( `cri-tools` ). Pass the endpoint, or set it once in `/etc/crictl.yaml` :

```
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps
```

## Typical runtime failures

| Symptom                                                                                        | Likely cause                                    | Lab                |
|------------------------------------------------------------------------------------------------|-------------------------------------------------|--------------------|
| <code>kubeadm init</code> preflight: container runtime is not running / CRI v1 not implemented | CRI plugin disabled in <code>config.toml</code> | <a href="#">03</a> |

| Symptom                                                                                  | Likely cause                                                  | Lab                                      |
|------------------------------------------------------------------------------------------|---------------------------------------------------------------|------------------------------------------|
| Node <code>NotReady</code> , kubelet log: cannot connect to <code>containerd.sock</code> | containerd stopped or crashing                                | 07                                       |
| Pods restart randomly under load, kubelet warnings about cgroups                         | cgroup driver mismatch                                        | Step 3 of the kubeadm lesson             |
| <code>ErrImagePull</code> / <code>ImagePullBackOff</code>                                | wrong image name/tag, no registry access, missing pull secret | <code>kubect1 describe pod</code> events |

## Check yourself



### 1. What is the CRI?

The gRPC API the kubelet uses to talk to any container runtime: run Pod sandboxes, pull images, start and list containers.



### 1. Do images built with Docker still run on Kubernetes after dockershim was removed?

Yes. Docker builds standard OCI images, which containerd and CRI-O run unchanged. Only Docker Engine as the node's runtime was dropped.



### 1. containerd is "active" but kubeadm says the runtime is not usable. What do you check?

Whether the CRI answers: `sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock version` , and whether `config.toml` lists `cri` under `disabled_plugins` .



### 1. Why does `docker ps` show nothing on a kubeadm node?

Docker is not installed; Kubernetes runs its containers through containerd. Use `crictl ps` .



### 1. Where is the `SystemdCgroup` setting in containerd 2.x?

Under `[plugins.'io.containerd.cri.v1.runtime'.containerd.runtimes.runc.options]` .

## 05 · What kubeadm does, step by step

Time: 25 minutes · The theory behind Steps 5 and 7 of the [kubeadm lesson](#).

### What kubeadm does, and does not do

| kubeadm does                                                              | kubeadm does not                                  |
|---------------------------------------------------------------------------|---------------------------------------------------|
| check that the machine is ready (preflight)                               | create machines or networks                       |
| create the cluster's certificate authority and every certificate          | install or configure the container runtime        |
| write the kubeconfig files for admin and components                       | install the kubelet (it comes from the package)   |
| write static Pod manifests for the control plane                          | install a CNI plugin                              |
| install CoreDNS and kube-proxy                                            | install add-ons (Ingress, storage, monitoring)    |
| create bootstrap tokens and join nodes                                    | make the control plane highly available by itself |
| upgrade a cluster ( <code>kubeadm upgrade</code> ) and renew certificates | back up etcd                                      |

It is a **bootstrapper**: it does the Kubernetes-specific setup correctly, and nothing more. That is why it is also used inside other tools (kind and Minikube call kubeadm internally).

### The phases of `kubeadm init`

`kubeadm init` is a fixed list of **phases**. You can run any of them alone with `kubeadm init phase <name>`, which is how troubleshooting [03](#) re-runs only the preflight checks.

| Phase                      | What happens                                                                                                                                                | What you can see afterwards                                                                                                                                              |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>preflight</code>     | checks CPUs ( $\geq 2$ ), memory ( $\geq 1700$ MB), swap, free ports, the container runtime through the CRI, required tools; pulls the control-plane images | <code>[ERROR ...]</code> lines if something is wrong; nothing is changed yet                                                                                             |
| <code>certs</code>         | creates the cluster CA, the etcd CA, the front-proxy CA and the certificates for API server, kubelet client, etcd peers, service-account signing keys       | <code>/etc/kubernetes/pki/</code>                                                                                                                                        |
| <code>kubeconfig</code>    | writes kubeconfig files with client certificates                                                                                                            | <code>/etc/kubernetes/admin.conf</code> , <code>super-admin.conf</code> , <code>kubelet.conf</code> , <code>controller-manager.conf</code> , <code>scheduler.conf</code> |
| <code>etcd</code>          | writes the static Pod manifest for a local etcd                                                                                                             | <code>/etc/kubernetes/manifests/etcd.yaml</code>                                                                                                                         |
| <code>control-plane</code> | writes static Pod manifests for the API server, controller manager and scheduler                                                                            | <code>/etc/kubernetes/manifests/kube-*.yaml</code>                                                                                                                       |

| Phase              | What happens                                                                                       | What you can see afterwards                                         |
|--------------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| kubelet-start      | writes the kubelet's configuration and (re)starts it; the kubelet now starts the static Pods       | <code>/var/lib/kubelet/config.yaml</code>                           |
| wait-control-plane | waits until the API server answers healthy                                                         | the long pause in the output                                        |
| upload-config      | stores the kubeadm and kubelet configuration in ConfigMaps, for joins and upgrades                 | <code>kubect1 -n kube-system get cm kubeadm-config</code>           |
| upload-certs       | (only with <code>--upload-certs</code> ) shares control-plane certificates encrypted, for HA joins | Secret <code>kubeadm-certs</code>                                   |
| mark-control-plane | labels and taints the node so normal Pods are not scheduled there                                  | taint <code>node-role.kubernetes.io/control-plane:NoSchedule</code> |
| bootstrap-token    | creates a join token and the RBAC rules that let new nodes use it                                  | <code>kubeadm token list</code>                                     |
| kubelet-finalize   | switches the kubelet to a rotating client certificate                                              |                                                                     |
| addon              | installs CoreDNS and kube-proxy                                                                    | Deployment <code>coredns</code> , DaemonSet <code>kube-proxy</code> |

## Static Pods

The control plane runs as **static Pods**: the kubelet reads YAML files in `/etc/kubernetes/manifests` and starts those Pods itself, without the API server. That solves the chicken-and-egg problem (the API server cannot schedule itself). To change an API server flag, edit its manifest: the kubelet notices and restarts it. The Pods you see in `kubect1 get pods -n kube-system` with the node name as suffix ( `kube-apiserver-k8s-cp` ) are "mirror" copies.

## Files on a control plane after init

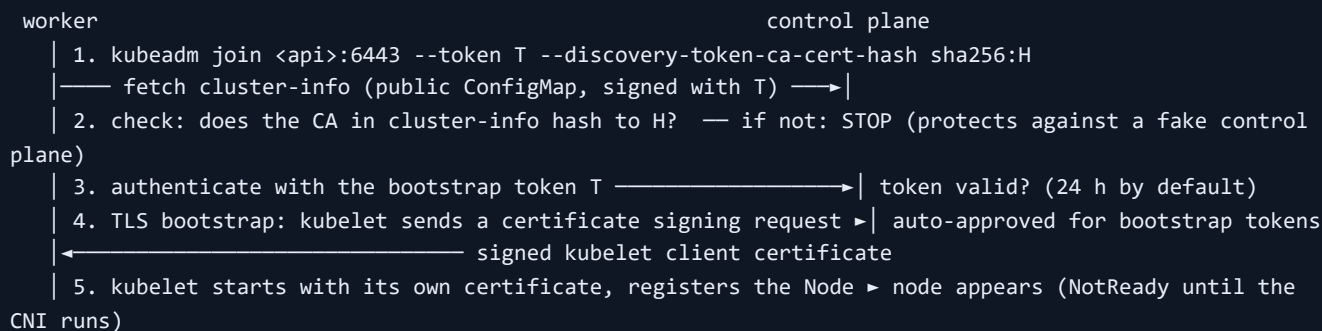
```

/etc/kubernetes/
├─ admin.conf           cluster-admin kubeconfig (copied to ~/.kube/config in the lesson)
├─ super-admin.conf     break-glass kubeconfig (bypasses RBAC); keep it safe
├─ kubelet.conf         controller-manager.conf  scheduler.conf
├─ manifests/          etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-
scheduler.yaml
├─ pki/                ca.crt  ca.key  apiserver.*  apiserver-kubelet-client.*  front-proxy-*
sa.key  sa.pub
├─ etcd/               ca.*  server.*  peer.*  healthcheck-client.*
/var/lib/etcd/         etcd's data: the whole cluster state
/var/lib/kubelet/      kubelet configuration, Pod volumes

```

The `.key` files are the cluster's crown jewels: anyone with `ca.key` can create an admin certificate.

## How `kubeadm join` works

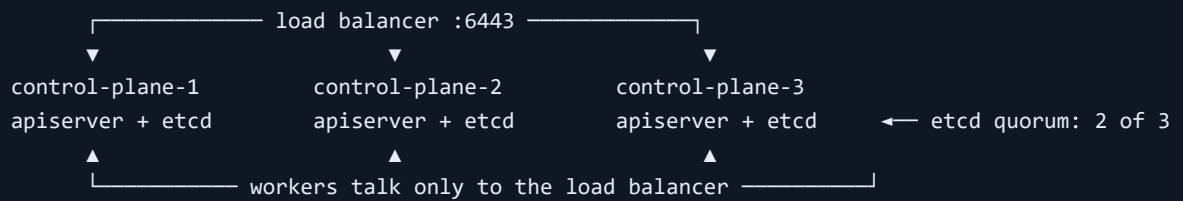


- The **token** proves the worker is allowed to join. It expires: always create a fresh command with `kubeadm token create --print-join-command` (troubleshooting 04).
- The **CA cert hash** proves to the worker that it is talking to the real control plane.
- After **TLS bootstrapping**, the token is no longer used: the kubelet has its own certificate, which it rotates.

## Production is different

The lesson builds **one** control-plane node. If it fails, the cluster cannot be managed. Production clusters differ:

| Topic                  | Learning cluster                     | Production                                                                                                                                                                                                                               |
|------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Control plane          | 1 node                               | <b>3</b> (or 5) control-plane nodes, so etcd keeps a majority when one fails                                                                                                                                                             |
| API endpoint           | the node's IP                        | a <b>load balancer</b> (or virtual IP) in front of all API servers, set with <code>kubeadm init - -control-plane-endpoint &lt;lb-dns&gt;:6443</code> from the start (it cannot be added later without pain)                              |
| Joining control planes | n/a                                  | <code>kubeadm init --upload-certs</code> , then <code>kubeadm join ... --control-plane --certificate-key &lt;key&gt;</code>                                                                                                              |
| etcd                   | stacked (on the control-plane nodes) | stacked, or <b>external etcd</b> on dedicated machines for large clusters                                                                                                                                                                |
| Certificates           | valid 1 year, CA 10 years            | monitor with <code>kubeadm certs check-expiration</code> ; <code>kubeadm upgrade</code> renews them; or <code>kubeadm certs renew all</code>                                                                                             |
| Upgrades               | rebuild                              | one minor version at a time: <code>kubeadm upgrade plan</code> , <code>kubeadm upgrade apply v1.37.x</code> on the first control plane, <code>kubeadm upgrade node</code> on the others, then drain, upgrade kubelet, uncordon each node |
| Backups                | none                                 | scheduled <code>etcdctl snapshot save</code> , stored off the cluster, restore tested                                                                                                                                                    |
| Machines               | prepared by hand or a script         | image building (Packer) + configuration management (Ansible) or Cluster API                                                                                                                                                              |
| Security               | defaults                             | API server audit logging, encryption of Secrets at rest, Pod Security admission, network policies, restricted SSH                                                                                                                        |



## Check yourself



### 1. Why does the control plane run as static Pods?

The kubelet can start static Pods from files without an API server, so the API server itself can run as a Pod. That solves the bootstrap problem.



### 1. kubeadm init failed at preflight. Do you need to reset the machine?

No. Preflight runs before anything is changed. Fix the cause and run `kubeadm init` again.



### 1. What do the token and the CA hash in a join command each protect?

The token proves to the cluster that the node may join. The CA hash proves to the node that it is talking to the real control plane.



### 1. Why must `--control-plane-endpoint` be set from the beginning in production?

It is written into every certificate and kubeconfig. Pointing it at a load balancer from the start lets you add control-plane nodes later; changing it afterwards means reissuing certificates and configs.



### 1. Which file contains everything about your cluster's state, and how do you protect it?

etcd's data in `/var/lib/etcd`. Protect it with regular `etcdctl snapshot save` backups stored off the cluster, and practise restoring them.

## 06 · CNI and Pod networking

Time: 20 minutes · Background for Step 6 of the [kubeadm lesson](#) and for [troubleshooting 02](#).

### The Kubernetes network model

Kubernetes sets three rules, and leaves the implementation to a plugin:

1. **Every Pod gets its own IP address.** Containers in one Pod share it (they reach each other on `localhost`).
2. **Every Pod can reach every other Pod**, on any node, **without NAT**: the IP a Pod sees for itself is the IP others use to reach it.
3. **Agents on a node** (kubelet, system daemons) can reach all Pods on that node.

Services (stable virtual IPs in front of Pods) are a separate layer, done by kube-proxy. Network policies (firewall rules between Pods) are optional and must be supported by the plugin.

### What CNI is

The **Container Network Interface** is a small specification plus plugins. When the runtime creates a Pod's network namespace, it calls the CNI plugin configured in `/etc/cni/net.d/`, with binaries in `/opt/cni/bin/`. The plugin:

- creates the Pod's network interface ( `eth0` inside the Pod) and connects it to the node
- assigns the Pod an IP from the node's range (IPAM)
- makes sure other nodes know how to reach that range

### Why a node is NotReady without a CNI

Right after `kubeadm init`, `kubectl describe node` in the lesson shows (real output, Kubernetes 1.37):

```
container runtime network not ready: NetworkReady=false reason:NetworkPluginNotReady
message:Network plugin returns error: cni plugin not initialized
```

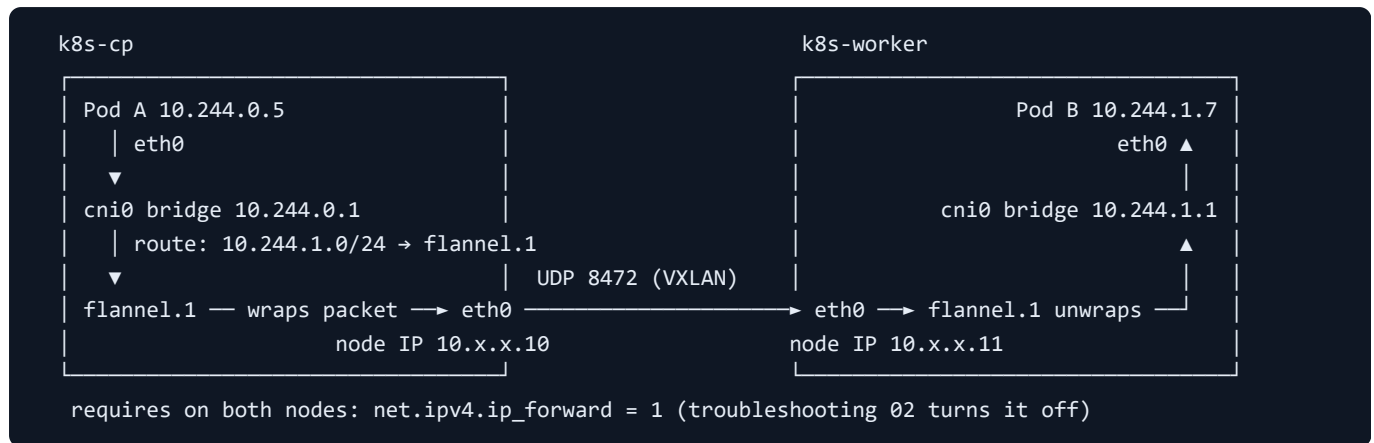
The kubelet will not mark a node `Ready` until the runtime reports a working network, because Pods started there would have no connectivity. CoreDNS stays `Pending` for the same reason. Installing Flannel fixes both within seconds.

### Flannel, as used in this course

**Flannel v0.28.9** is the simplest CNI: an overlay network, no network policies.

- Cluster Pod range **10.244.0.0/16** (that is why `kubeadm init --pod-network-cidr=10.244.0.0/16`).
- Each node gets a **/24** of it: for example `10.244.0.0/24` on the control plane, `10.244.1.0/24` on the worker.
- On each node: the bridge `cni0` connects local Pods; the VXLAN device `flannel.1` wraps packets for other nodes in UDP (port 8472).

A packet from a Pod on the control plane to a Pod on the worker:



## Other CNI plugins

| Plugin         | How it works                                                                                                       | Network policies      | Typical use                                                   |
|----------------|--------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------------------------------|
| Flannel        | VXLAN overlay                                                                                                      | no                    | learning, small clusters, k3s default                         |
| Calico         | routed (BGP) or overlay (VXLAN/IP-in-IP), or eBPF data plane                                                       | yes                   | most self-managed production clusters; MicroK8s default       |
| Cilium         | eBPF in the kernel; can also replace kube-proxy                                                                    | yes, also on layer 7  | large clusters, observability (Hubble), service mesh features |
| Amazon VPC CNI | no overlay: each Pod gets a <b>real IP from the VPC subnet</b> , as a secondary IP on the node's network interface | with the policy agent | EKS default ( <code>aws-node</code> Pods in the lesson)       |

## How to choose

Managed cloud cluster? → use the provider's CNI (EKS: VPC CNI), unless you have a reason not to  
Need network policies? → Calico or Cilium (not Flannel)  
Very large / need eBPF features → Cilium  
Learning, lab, minimal → Flannel

Two practical consequences:

- The Pod range must **not overlap** with your node network or VPC, or packets go to the wrong place.
- On EKS with the VPC CNI, every Pod uses a VPC IP: small subnets limit how many Pods you can run, and each instance type has a maximum number of IPs (hence a maximum number of Pods per node).

## Check yourself



1. What are the three rules of the Kubernetes network model?

Every Pod has its own IP; all Pods can reach all Pods without NAT; node agents can reach the Pods on their node.



### 1. Why is a fresh kubeadm node NotReady?

No CNI plugin is installed, so the runtime reports `NetworkPluginNotReady` and the kubelet does not mark the node Ready.



### 1. What are `cni0` and `flannel1.1`?

`cni0` is the bridge connecting the Pods on one node; `flannel1.1` is the VXLAN device that wraps packets for Pods on other nodes.



### 1. What is special about Pod IPs on EKS with the VPC CNI?

They are real VPC addresses from the node's subnet, not an overlay. Anything in the VPC can route to them, and subnet size limits the number of Pods.



### 1. You need network policies. Is Flannel a good choice?

No. Flannel does not enforce network policies. Choose Calico or Cilium.

## 07 · The four installation methods

Time: 15 minutes · An overview before the hands-on lessons. The detailed side-by-side comparison is in [docs/13](#).

All four methods give you the **same Kubernetes API**: the same `kubectl` commands and the same YAML work everywhere. They differ in who runs the control plane, where the nodes are, and how much you must do yourself.

| more control, more work |                         | less control, less work |                      |
|-------------------------|-------------------------|-------------------------|----------------------|
| kubeadm                 | MicroK8s                | Minikube                | Amazon EKS           |
| you build every layer   | one snap, everything in | one command, a cluster  | AWS runs the control |
| plane,                  | it, real Linux machines | in a container/VM       | you run (or rent)    |
| on real Linux machines  |                         |                         |                      |
| workers                 | self-managed / edge     | local only              | managed, production  |
| self-managed            |                         |                         |                      |
| cloud                   |                         |                         |                      |

### kubeadm

- **What:** the official bootstrapper. You prepare Linux machines (prerequisites, containerd, packages), then `kubeadm init` creates the control plane and `kubeadm join` adds nodes. You install the CNI yourself.
- **Components:** control plane as static Pods; etcd as a static Pod; kubelet and containerd as systemd services.
- **Who uses it:** platform teams running on-premises or bare metal, anyone preparing for the CKA exam, and tools built on top of it (Cluster API, kind, Minikube).
- **Pros:** standard, transparent, every layer visible; you learn how a cluster really works.
- **Cons:** you own everything: HA, upgrades, certificates, backups, the CNI.
- **Lesson:** [kubeadm on Ubuntu](#) (Kubernetes v1.37, containerd 2.x, Flannel v0.28.9).

### Minikube

- **What:** a local, throw-away cluster on your computer, usually as one Docker container (the "node"), also as a VM.
- **Components:** everything inside the node container (it uses kubeadm internally); add-ons with one command.
- **Who uses it:** developers testing manifests and Helm charts, trainers, CI jobs.
- **Pros:** one command, runs on Windows, macOS and Linux, profiles for several clusters, easy reset.
- **Cons:** not for production; limited by your laptop's resources; networking differs from real clusters.
- **Lesson:** [Minikube](#) (v1.39.0).

### MicroK8s

- **What:** Canonical's lightweight Kubernetes, packaged as one **snap**. Installs on any Linux with snap support, joins several machines into a cluster with `microk8s add-node`.

- **Components:** control plane in one process (**kubelite**), datastore **dqlite** instead of etcd, containerd and Calico included; add-ons with `microk8s enable`.
- **Who uses it:** edge and IoT, small self-managed clusters, developers on Ubuntu, CI.
- **Pros:** one command install, automatic security updates within a track, small footprint, HA with three nodes.
- **Cons:** snap-specific layout and commands ( `microk8s kubectl` ), less common in large enterprises.
- **Lesson:** [MicroK8s](#) (track `1.36/stable` , the newest stable track at the time of writing).

## Amazon EKS

- **What:** AWS's managed Kubernetes. AWS runs and scales the control plane across availability zones; you choose the worker nodes (managed node groups, Fargate, or EKS Auto Mode).
- **Components:** control plane and etcd invisible to you; workers are EC2 instances; Amazon VPC CNI, CoreDNS and kube-proxy as add-ons; IAM controls access.
- **Who uses it:** companies running production workloads on AWS.
- **Pros:** no control plane to operate, integrated with AWS (IAM, load balancers, EBS volumes), upgrades by API.
- **Cons:** costs money from the first minute (control plane per hour, nodes, NAT gateway, load balancers); AWS networking and IAM knowledge required; resources must be deleted carefully.
- **Lesson:** [Amazon EKS with eksctl](#) (EKS 1.37, eu-central-1, with a cost table and a verified teardown in [eks/cleanup.md](#)).

## Which one when?

| Situation                                             | Choose                                                                   |
|-------------------------------------------------------|--------------------------------------------------------------------------|
| "I want to understand how Kubernetes is put together" | kubeadm                                                                  |
| "I need a cluster on my laptop in 2 minutes"          | Minikube                                                                 |
| "Small cluster on a few Linux boxes, or at the edge"  | MicroK8s                                                                 |
| "Production on AWS"                                   | EKS                                                                      |
| "Production on my own hardware"                       | kubeadm (with HA, automation and backups), or a distribution built on it |

## The order of this course

kubeadm (learn every layer) → troubleshooting labs 01-07 → Minikube → troubleshooting lab 08 → MicroK8s → EKS

kubeadm comes first on purpose: once you have built every layer yourself, the other three are easy to understand, because you know what they are doing for you.

## Check yourself



### 1. Which methods let you see the API server as a Pod?

kubeadm and Minikube (static Pods). MicroK8s runs it inside kubelite; EKS runs it outside your account.



### 1. Which method costs money while it runs, and why does that matter?

EKS: the control plane, EC2 nodes, NAT gateway and load balancers are billed per hour, so you must delete everything when you are done and verify it is gone.



### 1. What does MicroK8s use instead of etcd?

dqlite, a distributed SQLite.



### 1. A team wants production Kubernetes in their own data centre. Which method fits?

kubeadm with a highly available control plane, automation and backups (or a distribution that builds on it). MicroK8s is an option for smaller clusters.

# 08 · How Minikube works inside

Time: 15 minutes · Read it before or after the [Minikube lesson](#).

Minikube looks like magic: one command, and a whole Kubernetes cluster runs on your laptop. This chapter opens the box. Everything inside is the same Kubernetes you build by hand in the [kubeadm lesson](#), packed so it fits in a single machine (or a single container).

## The big idea: one "node" that Minikube creates for you

A Kubernetes node is normally a whole machine. Minikube creates that machine for you, using a **driver**:

| Driver                                                 | What the node is                  | Typical use                                                  |
|--------------------------------------------------------|-----------------------------------|--------------------------------------------------------------|
| <code>docker</code> (default when Docker is installed) | a Docker <b>container</b>         | Linux, Windows and macOS with Docker Desktop / Docker Engine |
| <code>podman</code>                                    | a Podman container                | Linux without Docker (still marked experimental by Minikube) |
| <code>hyperv</code>                                    | a Hyper-V virtual machine         | Windows Pro/Enterprise without Docker                        |
| <code>kvm2</code>                                      | a KVM virtual machine             | Linux, when you want a real VM                               |
| <code>qemu</code>                                      | a QEMU virtual machine            | macOS on Apple silicon, Linux                                |
| <code>none</code> / <code>ssh</code>                   | your own machine, or a remote one | special cases; not for learning                              |

In this course we use `--driver=docker`. Then the "node" is a container named `minikube`, and **inside** that container run `systemd`, `containerd`, the `kubelet`, and the control plane:

```
Your computer
├─ Docker Engine
│   └─ container "minikube" (image: kicbase, the "Kubernetes in container" base image)
│       ├── systemd                starts the services below, like on a real Linux machine
│       ├── containerd             the container runtime (CRI) for the cluster
│       ├── kubelet                 the node agent
│       └─ containers started by containerd, inside the minikube container:
│           ├── kube-apiserver, etcd, kube-scheduler, kube-controller-manager (static Pods)
│           ├── kube-proxy, CoreDNS
│           ├── storage-provisioner      (Minikube's own: creates volumes for PersistentVolumeClaims)
│           └─ your Pods (web, ...)
```

So there are **containers inside a container**. That is why, in the lesson, `docker ps` shows only one container, `minikube`, while `minikube ssh -- sudo crictl ps` shows all the Kubernetes components.

## What `minikube start` does

1. Pick the driver and check resources (CPUs, memory, free disk; refuses below the minimum)
2. Download the kicbase image + a preloaded image bundle for the Kubernetes version (a tarball with all Kubernetes images, so the start is fast)
3. Create the node (docker run ... kicbase)
4. Run kubeadm INSIDE the node (yes: Minikube uses kubeadm to build the control plane)
5. Install a CNI if needed, CoreDNS, the storage provisioner
6. Write a context "minikube" into ~/.kube/config and make it the current context

Step 4 is worth remembering: the certificates, static Pod manifests and /etc/kubernetes layout inside the node are the same as on a kubeadm cluster. What you learn in one applies to the other.

## Where things live

| What                                                               | Where                                                                                    |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Minikube's own files (cached images, profiles, certificates, logs) | ~/minikube                                                                               |
| The kubeconfig entry (context and cluster named after the profile) | ~/kube/config                                                                            |
| The node                                                           | the Docker container minikube (or one container per node: multinode, multinode-m02, ...) |
| Inside the node: Kubernetes configuration                          | /etc/kubernetes (open it with minikube ssh)                                              |

## Profiles: several clusters side by side

A **profile** is one complete cluster with its own name. The default profile is called minikube. In the lesson you create a second one:

```
minikube start -p multinode --nodes 2 --driver=docker --cpus=2 --memory=2200
kubectl --context multinode get nodes
minikube profile list
minikube delete -p multinode
```

Every minikube command takes -p <profile>. Every profile is also a kubectl **context**, so you switch with kubectl config use-context <name> OR --context.

## Multi-node

--nodes 2 creates a second container (multinode-m02) and joins it as a worker, again with kubeadm. Minikube installs a CNI (kindnet) so Pods on the two nodes can reach each other. It is good for seeing scheduling across nodes and DaemonSets. It is **not** high availability: both "nodes" share your one computer.

## Add-ons

Add-ons are optional components Minikube installs with one command: metrics-server, dashboard, ingress, registry and many more. minikube addons list shows them; minikube addons enable metrics-server installs one. They are ordinary Kubernetes objects; kubectl get pods -n kube-system shows them afterwards.

## Reaching your Services: `minikube service` and `minikube tunnel`

With the Docker driver on Windows and macOS, the node's IP lives inside Docker's own network and is not reachable from your computer. Minikube solves this in two ways:

| Command                                 | What it does                                                                | Use it for                                      |
|-----------------------------------------|-----------------------------------------------------------------------------|-------------------------------------------------|
| <code>minikube service web --url</code> | prints a URL (and on Windows/macOS keeps a port-forward open while it runs) | NodePort Services; used in the lesson           |
| <code>minikube tunnel</code>            | runs a route/proxy so <code>LoadBalancer</code> Services get an external IP | testing <code>type: LoadBalancer</code> locally |

On Linux with the Docker driver, `minikube ip` + the NodePort also works directly.

## How big should it be?

| Setting | Minimum                                                                       | Comfortable for this course            |
|---------|-------------------------------------------------------------------------------|----------------------------------------|
| CPUs    | 2                                                                             | 2                                      |
| Memory  | about 1.8 GB (Minikube refuses less, see <a href="#">troubleshooting 08</a> ) | 4096 MB ( <code>--memory=4096</code> ) |
| Disk    | about 20 GB free                                                              | 20 GB+                                 |

The memory goes to the node container and is shared by the control plane, system Pods and your workloads.

## Check yourself

### ▼ With the Docker driver, how many Docker containers does a one-node Minikube cluster create, and where do the Kubernetes components run?

One container, `minikube`. The components run as containers **inside** it, started by the containerd that runs inside the node container. `docker ps` on your computer only shows the node.

### ▼ Which tool does Minikube use internally to build the control plane?

kubeadm. That is why the layout inside the node ( `/etc/kubernetes` , static Pods, certificates) matches the kubeadm lesson.

### ▼ What is a profile, and how do you talk to a second profile with kubectl?

A complete, separately named cluster. Each profile is also a kubectl context of the same name: `kubectl --context multinode get nodes` , or `kubectl config use-context multinode` .

### ▼ Your NodePort Service works inside the cluster, but the node IP is not reachable from your Windows laptop. Why, and what do you use?

With the Docker driver on Windows/macOS the node lives in Docker's internal network. Use `minikube service web --url` (or `minikube tunnel` for LoadBalancer Services).

### ▼ Is a two-node Minikube cluster highly available?

No. Both nodes are containers on the same computer, and there is one control plane. It is useful to learn scheduling across nodes, not for availability.

## 09 · How MicroK8s works inside

Time: 15 minutes · Read it before or after the [MicroK8s lesson](#).

MicroK8s is Canonical's Kubernetes distribution packed as a single **snap** package. One `snap install` gives you a complete, conformant cluster on one Linux machine, and a few more commands turn several machines into a highly-available cluster. This chapter explains what is in that package and why it behaves the way it does.

### Snap packaging

A snap is a self-contained package: the application and everything it needs (here: Kubernetes, containerd, the CNI, the datastore) in one compressed, read-only image, mounted at `/snap/microk8s/<revision>`.

| Snap idea     | What it means for MicroK8s                                                                                                                   |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Track         | the Kubernetes minor version, e.g. <code>1.36</code>                                                                                         |
| Risk level    | <code>stable</code> , <code>candidate</code> , <code>beta</code> , <code>edge</code> ; the lesson uses <code>--channel=1.36/stable</code>    |
| Revision      | every update is a new revision; the previous one is kept, so <code>snap revert microk8s</code> rolls back                                    |
| Confinement   | MicroK8s is installed with <code>--classic</code> : it needs broad access to the host (networking, cgroups, mounts) like any Kubernetes node |
| Writable data | configuration and state live in <code>/var/snap/microk8s/current/</code> (e.g. the component arguments in <code>args/</code> )               |

### Updates happen on their own

snaptd refreshes installed snaps automatically (by default several times a day it checks for new revisions). Within a track, MicroK8s only receives **patch** releases (`1.36.x` → `1.36.y`); it never jumps to a new minor version unless you change the track ( `sudo snap refresh microk8s --channel=1.37/stable` ). On a production machine, control when refreshes happen with `sudo snap refresh --hold=... microk8s` or a refresh timer ( `snap set system refresh.timer=...` ), so a node does not restart its Kubernetes services in the middle of the day.

### Kubelite: the control plane in one process

On a kubeadm cluster the API server, scheduler and controller manager run as separate static Pods, and kube-proxy as a DaemonSet. MicroK8s runs them **together in one process**, `kubelite`, as a systemd service:

```

Ubuntu machine (k8s-micro)
├─ snap.microk8s.daemon-kubelite    kube-apiserver + kube-scheduler + kube-controller-manager
│                                   + kubelet + kube-proxy, all in ONE process
├─ snap.microk8s.daemon-containerd  the container runtime (its own containerd, separate from any
Docker)
├─ snap.microk8s.daemon-k8s-dqlite  the datastore (instead of etcd)
├─ snap.microk8s.daemon-cluster-agent joins and manages other nodes (microk8s add-node / join)
└─ Pods
    ├─ calico-node, calico-kube-controllers  the CNI
    ├─ coredns
    └─ your Pods (web, ...)

```

That is why the lesson's `systemctl list-units 'snap.microk8s.*'` lists services, and why `kubectl get pods -n kube-system` shows **no** `kube-apiserver` or `etcd` Pods on MicroK8s: they are not Pods here.

## dqlite instead of etcd

The cluster state (every object you create) is stored in **dqlite**: a distributed SQLite with Raft replication, made by Canonical. It plays exactly the role etcd plays in the [architecture](#). With one node it is a local database; with three or more nodes it is replicated automatically.

## Networking: Calico by default

MicroK8s enables **Calico** as its CNI out of the box, so a fresh one-node cluster is `Ready` without you applying a network plugin (compare the kubeadm lesson, where the node stays `NotReady` until you install Flannel, see [06 · CNI](#)).

## Add-ons

Most extras are one command away: `microk8s enable hostpath-storage, metrics-server, dns` (CoreDNS, enabled by default in current releases), `ingress, dashboard, registry, metallb, rbac, ...` `microk8s status` lists them as enabled or disabled. Under the hood an add-on is a script that applies Kubernetes manifests or Helm charts.

### `microk8s kubectl` and the alias

MicroK8s ships its own kubectl, matching the cluster version, as `microk8s kubectl`. You can:

| Option               | Command                                               | Note                                                      |
|----------------------|-------------------------------------------------------|-----------------------------------------------------------|
| use it directly      | <code>microk8s kubectl get nodes</code>               | always matches the cluster                                |
| alias it             | <code>sudo snap alias microk8s.kubectl kubectl</code> | done in the lesson; then plain <code>kubectl</code> works |
| use your own kubectl | <code>microk8s config &gt; ~/.kube/config</code>      | exports the admin kubeconfig, e.g. for a laptop           |

## Groups and permissions

MicroK8s commands need access to files that belong to root. Instead of typing `sudo` every time, add your user to the `microk8s` group ( `sudo usermod -a -G microk8s "$USER"` ). Group changes apply to **new** logins

only, which is why the lesson uses `newgrp microk8s` (or log out and in again). Membership in this group is effectively cluster-admin: grant it only to people who should administer the cluster.

## High availability with three or more nodes

```
on the first node:  microk8s add-node          → prints a "microk8s join <ip>:25000/<token>" command
on each new node:  microk8s join <ip>:25000/<token>
```

From three control-plane nodes on, MicroK8s turns on HA automatically: dqlite replicates to the voters and the API server runs on every node. `microk8s status` then shows `high-availability: yes`. Nodes joined with `--worker` run workloads only.

## Check yourself

### ▼ Why are there no kube-apiserver or etcd Pods on a MicroK8s cluster?

The control plane runs inside the single `kubelite` process (a systemd service), and the datastore is dqlite, run by its own `k8s-dqlite` service. Neither is a Pod.

### ▼ You installed with `--channel=1.36/stable`. Will snapd upgrade you to Kubernetes 1.37 overnight?

No. Automatic refreshes stay inside the track (1.36.x patch releases). Moving to 1.37 needs `snap refresh microk8s --channel=1.37/stable`. Patch refreshes do restart the services, so control their timing in production.

### ▼ Why is a fresh MicroK8s node Ready immediately, when a fresh kubeadm node is NotReady?

MicroK8s installs Calico as its CNI automatically. kubeadm installs no network plugin; you apply one yourself.

### ▼ After `usermod -a -G microk8s`, `microk8s status` still says permission denied. Why?

Group membership is read at login. Start a new login shell ( `newgrp microk8s` , or log out and in).

### ▼ How many nodes do you need for MicroK8s high availability?

Three or more control-plane nodes; HA switches on automatically when the third joins.

# 10 · Amazon EKS architecture

Time: 20 minutes · Read it before the [EKS lesson](#).

On kubeadm, Minikube and MicroK8s **you** run the control plane. On Amazon EKS (Elastic Kubernetes Service), AWS runs it for you, and you only bring the worker nodes (or let AWS run those too). This chapter shows where every piece lives, who pays for what, and how identity works, because that is where most EKS surprises come from.

## Two accounts: AWS's and yours



Key points:

- The control plane is **multi-AZ** and **not visible as machines**: `kubectl get nodes` lists only your workers. You cannot SSH into it, and you do not see `kube-apiserver` or `etcd` Pods in `kube-system`.
- AWS places **ENIs** (elastic network interfaces) of the control plane into your subnets, so the API server can reach the kubelets (for `kubectl logs`, `exec`, webhooks) inside your VPC.
- Our `cluster.yaml` puts the nodes in **private subnets** (`privateNetworking: true`): they have no public IP and reach the internet (image pulls, AWS APIs) through the single NAT gateway. Production uses one NAT gateway per AZ.

## Endpoint access

| Setting | Who can reach the API server                                                                                | Our lab                               |
|---------|-------------------------------------------------------------------------------------------------------------|---------------------------------------|
| public  | anyone on the internet (still needs valid AWS credentials + Kubernetes permission); can be limited to CIDRs | on, so kubectl works from your laptop |

| Setting          | Who can reach the API server                      | Our lab                                  |
|------------------|---------------------------------------------------|------------------------------------------|
| private          | only from inside the VPC (and connected networks) | on, so node traffic stays inside the VPC |
| public + private | both                                              | ✓                                        |

## Where the worker nodes come from

| Option                                                        | What you manage                                                                                                                | When                                                                |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Managed node group</b><br>(our lab: <code>workers</code> ) | instance type, size, scaling limits; AWS creates the Auto Scaling group + launch template, and handles draining during updates | the usual default                                                   |
| <b>EKS Auto Mode</b>                                          | almost nothing: AWS picks, launches, patches and replaces nodes (Karpenter-based), plus storage and load balancing             | teams who want the least operations work; costs a fee on top of EC2 |
| <b>Fargate</b>                                                | nothing at node level: each Pod runs in its own AWS-managed micro-VM                                                           | small or bursty workloads; no DaemonSets                            |
| <b>Self-managed nodes</b>                                     | everything: your own Auto Scaling group, AMI, bootstrap, upgrades                                                              | special AMIs or OS requirements                                     |

Our nodes run **Amazon Linux 2023** (the EKS-optimised AMI) with **containerd** and the kubelet, exactly the components from [02 · Architecture](#). In the lesson, `kubectl get nodes -o wide` shows the OS image and `containerd://2.x`.

## Pod networking: the Amazon VPC CNI

EKS uses the **VPC CNI** ( `aws-node` DaemonSet) by default. Unlike Flannel's overlay ([06 · CNI](#)), every Pod gets a **real IP address from your VPC subnet** (in the lesson, Pods have `10.50.x.x` addresses, like the nodes). Consequences:

- no overlay: Pods are directly routable inside the VPC, and security groups and VPC flow logs see Pod IPs;
- the number of Pods per node is limited by how many ENIs and IPs the instance type supports;
- subnets must be big enough for nodes **and** Pods (a /16 VPC leaves plenty of room).

## Identity: three different questions

- |                                                 |                            |                                                               |
|-------------------------------------------------|----------------------------|---------------------------------------------------------------|
| 1. What may the CONTROL PLANE do in my account? | → cluster IAM role         | ( <code>eksctl-k8s-from-zero-cluster-ServiceRole-...</code> ) |
| 2. What may the NODES do?                       | → node IAM role            | (pull images from ECR, attach ENIs, join the cluster)         |
| 3. Which HUMANS / CI may use kubectl?           | → access entries           | (IAM principal → Kubernetes permissions)                      |
| 4. What may a POD do in AWS?                    | → EKS Pod Identity or IRSA | (an IAM role for one service account)                         |

**Access entries vs aws-auth.** Kubernetes itself knows nothing about IAM. EKS maps IAM identities to Kubernetes users:

| Mode                                                                          | How                                                                                                                               | Status                                  |
|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <code>aws-auth</code> ConfigMap                                               | a YAML list in <code>kube-system</code> mapping IAM ARNs to groups                                                                | legacy; one typo can lock everybody out |
| <b>access entries</b><br>( <code>authenticationMode: API</code> ,<br>our lab) | managed through the EKS API / console / eksctl, with AWS-managed access policies (e.g. <code>AmazonEKSClusterAdminPolicy</code> ) | current recommendation                  |

The IAM identity that creates the cluster gets an admin access entry automatically. Anyone else, including a console user, needs an access entry before they can see Kubernetes objects. You can see this in the lesson's console tour: the read-only console user sees the cluster's AWS settings, but the **Resources** and **Compute** → **Nodes** views say "Unauthorized", because that user has no access entry.

kubectl authenticates with your **AWS** credentials: the kubeconfig entry runs `aws eks get-token` (or `eksctl`) on every request and sends a short-lived token.

## Add-ons

EKS add-ons are AWS-managed versions of cluster components that AWS keeps compatible with the Kubernetes version: `vpc-cni` , `kube-proxy` , `coredns` (installed by default) and optional ones such as `metrics-server` , `eks-pod-identity-agent` , `aws-ebs-csi-driver` . eksctl v0.231.0 also installed `metrics-server` in the lesson's run.

## What eksctl builds: CloudFormation stacks

eksctl does not call the EKS API piece by piece; it writes **CloudFormation** templates and lets CloudFormation create (and later delete) everything as a unit:

| Stack                                               | Contains                                                                                                                   |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>eksctl-k8s-from-zero-cluster</code>           | VPC, subnets, route tables, internet gateway, NAT gateway + Elastic IP, security groups, cluster IAM role, the EKS cluster |
| <code>eksctl-k8s-from-zero-nodegroup-workers</code> | node IAM role, launch template, managed node group (→ Auto Scaling group → EC2 instances + EBS volumes)                    |

Two things are **not** in these stacks: load balancers created by `Service type: LoadBalancer` (created by Kubernetes) and volumes created by `PersistentVolumeClaims`. Delete those from Kubernetes **before** deleting the cluster (12 · Cleanup).

## Support periods and versions

EKS 1.37 has **standard support** for about 14 months after its release on EKS, then optional **extended support** for another 12 months at a much higher control-plane price. The cluster's *upgrade policy* decides what happens at the end of standard support: stay on extended support (and pay for it) or be upgraded automatically. Check this setting on every cluster you own (console → cluster → Overview → Kubernetes version settings).

## What it costs

| Component             | Billed                                                                   | Note for our lab                           |
|-----------------------|--------------------------------------------------------------------------|--------------------------------------------|
| EKS control plane     | per cluster per hour (standard support: \$0.10/h at the time of writing) | the same for a tiny or a huge cluster      |
| EC2 worker nodes      | per instance per second                                                  | 2 × t3.medium                              |
| NAT gateway           | per hour + per GB processed                                              | runs as long as the VPC exists             |
| Public IPv4 addresses | per address per hour                                                     | NAT gateway EIP, load balancer             |
| Load balancer         | per hour + per capacity unit                                             | only while Service <code>web</code> exists |
| EBS volumes           | per GB-month                                                             | 2 × 20 GB gp3                              |
| Data transfer         | per GB out to the internet / across AZs                                  | small for a lab                            |

The estimate in `eks/cluster.yaml` is **about \$0.25–0.30 per hour** for the whole lab in eu-central-1. Prices change and differ by region: always check the [AWS pricing pages](#) and set an AWS Budget alarm before you create anything.

## Check yourself

### ▼ Why does `kubectl get nodes` on EKS show only your two workers?

The control plane runs in an AWS-managed account, not as nodes in your cluster. It reaches your VPC through ENIs.

### ▼ What is the practical difference between Pod IPs on EKS (VPC CNI) and on the kubeadm lab (Flannel)?

On EKS, Pods get real IPs from the VPC subnets (directly routable, limited by ENI/IP capacity per instance). Flannel gives Pods addresses from a separate overlay range (10.244.0.0/16) and encapsulates traffic between nodes.

### ▼ A colleague can see your cluster in the console but every Kubernetes view says "Unauthorized". What is missing?

An EKS access entry (with an access policy) for their IAM principal. IAM permissions to the EKS API alone do not give Kubernetes permissions.

### ▼ Which resources do NOT disappear when CloudFormation deletes the `eksctl` stacks, and why?

Load balancers from `Service` type: `LoadBalancer` and volumes from `PersistentVolumeClaims`. Kubernetes controllers created them through AWS APIs, so they are not part of the stacks. Delete them from Kubernetes first.

### ▼ Why did we put the nodes in private subnets, and what do they need to pull images?

So they have no public IP and cannot be reached from the internet. They need outbound access, provided here by the NAT gateway (or VPC endpoints for ECR/S3 in a locked-down setup).

# 11 · The cluster verification checklist

Time: 15 minutes · Used at the end of every installation lesson.

"The install finished without errors" is not the same as "the cluster works". Every installation in this course ends with the **same eight checks**, in the same order, from the bottom (nodes) to the top (an application reachable from outside). If one fails, the [troubleshooting labs](#) show how to find out why.

|   |                                                 |                                            |
|---|-------------------------------------------------|--------------------------------------------|
| 8 | kubectl and cluster versions are compatible     | ← your tools                               |
| 7 | Pods on different nodes can reach each other    | ← CNI across nodes                         |
| 6 | a Service is reachable from outside the cluster | ← kube-proxy / NodePort / load balancer    |
| 5 | a test Deployment runs 2 replicas               | ← scheduler, kubelet, runtime, image pulls |
| 4 | cluster DNS resolves Service names              | ← CoreDNS                                  |
| 3 | the API server answers and is healthy           | ← control plane                            |
| 2 | the system Pods are Running                     | ← kube-system                              |
| 1 | every node is Ready                             | ← kubelet + runtime + CNI on each node     |

## The checks

| # | Check                                                                                                                                    | Pass when                                                                                                                                           |
|---|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>kubectl get nodes -o wide</code>                                                                                                   | every node <code>Ready</code> , the expected version, <code>containerd://...</code> runtime                                                         |
| 2 | <code>kubectl get pods -A</code>                                                                                                         | all Pods <code>Running</code> (or <code>Completed</code> ), no <code>CrashLoopBackOff</code> , <code>Pending</code> , <code>ImagePullBackOff</code> |
| 3 | <code>kubectl cluster-info</code> and <code>kubectl get --raw='/readyz?verbose'</code>                                                   | the control plane URL is printed; <code>readyz</code> check passed                                                                                  |
| 4 | <code>kubectl run dnstest --image=busybox:1.37 --rm -i --restart=Never --quiet -- nslookup kubernetes.default.svc.cluster.local</code>   | an <code>Address:</code> line with the Service IP                                                                                                   |
| 5 | <code>kubectl create deployment web --image=nginx:1.30-alpine --replicas=2</code> and <code>kubectl rollout status deployment/web</code> | <code>successfully rolled out</code> , <code>2/2</code> ready                                                                                       |
| 6 | expose <code>web</code> and <code>curl</code> it from outside                                                                            | <code>&lt;title&gt;Welcome to nginx!&lt;/title&gt;</code>                                                                                           |
| 7 | <code>curl</code> a Pod IP that lives on <b>another</b> node                                                                             | the nginx title again                                                                                                                               |
| 8 | <code>kubectl version</code>                                                                                                             | client within one minor version of the server (e.g. 1.36–1.38 for a 1.37 cluster)                                                                   |

Instead of `kubectl create deployment`, you can also apply the ready-made manifests in [examples/nginx](#) (same Deployment, plus health checks and resource requests).

## The same checks on each cluster type

The commands are the ones used in the lessons. `kubectl` is the same everywhere; only how you reach the Service from outside, and how you open a shell on a node, differ.

| # | kubeadm (lesson)                                                                                                | Minikube (lesson)                                                                 | MicroK8s (lesson)                                                                                                 | EKS (lesson)                                                                                                                            |
|---|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <code>kubectl get nodes -o wide</code> (2 nodes)                                                                | <code>kubectl get nodes -o wide</code> (1 node)                                   | <code>microk8s kubectl get nodes -o wide</code> or <code>kubectl ...</code> after the alias                       | <code>kubectl get nodes -o wide</code> (2 workers, no control-plane nodes)                                                              |
| 2 | <code>kubectl get pods -A</code> : apiserver, etcd, scheduler, controller-manager, kube-proxy, CoreDNS, Flannel | <code>kubectl get pods -A</code> : same control-plane Pods + storage-provisioner  | <code>kubectl get pods -A</code> : Calico, CoreDNS (control plane is the <code>kubelite</code> service, not Pods) | <code>kubectl get pods -A</code> : aws-node, kube-proxy, CoreDNS, metrics-server (no control-plane Pods)                                |
| 3 | <code>kubectl cluster-info</code> , <code>kubectl get --raw='/readyz?verbose'</code>                            | <code>kubectl cluster-info</code> , <code>minikube status</code>                  | <code>kubectl cluster-info</code> , <code>microk8s status --wait-ready</code>                                     | <code>kubectl cluster-info</code> , <code>aws eks describe-cluster ... --query cluster.status</code> → <code>ACTIVE</code>              |
| 4 | <code>nslookup</code> from a busybox Pod                                                                        | same                                                                              | same                                                                                                              | same                                                                                                                                    |
| 5 | <code>web</code> , 2 replicas                                                                                   | same                                                                              | same                                                                                                              | same                                                                                                                                    |
| 6 | <code>--type=NodePort</code> , <code>curl http://&lt;worker-ip&gt;:&lt;node-port&gt;</code>                     | <code>--type=NodePort</code> , <code>curl "\$(minikube service web --url)"</code> | <code>--type=NodePort</code> , <code>curl http://127.0.0.1:&lt;node-port&gt;</code> on the VM                     | <code>--type=LoadBalancer</code> , <code>curl http://&lt;elb-hostname&gt;</code> (allow 1–3 minutes for DNS)                            |
| 7 | <code>curl</code> a Pod IP from the control plane                                                               | needs <code>--nodes 2</code> (one node: skip)                                     | one node: skip (multi-node: from another node)                                                                    | Pods have VPC IPs on both nodes: <code>kubectl get pods -l app=web -o wide</code> , then <code>curl</code> from a Pod on the other node |
| 8 | <code>kubectl version</code>                                                                                    | <code>kubectl version</code>                                                      | <code>kubectl version</code>                                                                                      | <code>kubectl version</code> (server shows <code>v1.37.x-eks-...</code> )                                                               |

## Why this order?

Each check depends on the ones below it. If nodes are `NotReady` (1), Pods will not schedule (5), so there is no point debugging the Service (6) yet. Always fix the **lowest** failing check first.

| First failing check | Look at                                                           |
|---------------------|-------------------------------------------------------------------|
| 1                   | <a href="#">Troubleshooting 01, 07</a>                            |
| 2                   | <a href="#">Troubleshooting 06</a>                                |
| 3                   | <a href="#">Troubleshooting 05</a>                                |
| 4                   | CoreDNS Pods and their logs ( <a href="#">06</a> )                |
| 5                   | <code>kubectl describe pod</code> : events ( <a href="#">08</a> ) |
| 6, 7                | <a href="#">Troubleshooting 02</a> , firewall/security groups     |

## Clean up the test workload

```
kubectl delete service web
kubectl delete deployment web
```

On EKS, deleting the Service also deletes the AWS load balancer; do it **before** deleting the cluster ([12 · Cleanup](#)).

## Check yourself

### ▼ Nodes are Ready and all system Pods run, but `curl` to the NodePort times out. Which checks narrow it down?

Check 5 (are the `web` Pods Running and Ready?), then `kubectl get endpoints web` (does the Service have Pod IPs?), then check 7 (Pod-to-Pod across nodes). If Pods answer directly but not through the NodePort, look at kube-proxy and firewalls.

### ▼ Why is check 7 the one that catches a broken CNI that check 5 may miss?

Both replicas can be Running and even reachable on their own node while traffic **between** nodes fails (overlay or routing broken). Only a request to a Pod on another node proves cross-node networking.

### ▼ Your `kubectl` is v1.33 and the cluster v1.37. Is that OK?

No. `kubectl` is supported within one minor version of the API server (1.36–1.38 here). Install a matching `kubectl`.

### ▼ On EKS the Service shows `<pending>` under EXTERNAL-IP for a minute. Is the check failed?

Not yet. AWS creates the load balancer and its DNS name takes a few minutes to resolve. Retry; investigate events ( `kubectl describe service web` ) only if it stays pending.

# 12 · Cleanup and reset

Time: 15 minutes · Every lesson has its own cleanup page; this chapter explains the ideas behind them.

Creating a cluster is half the job. Removing it **completely** is the other half:

- **Cost.** A forgotten EKS cluster keeps billing every hour: control plane, nodes, NAT gateway, load balancer, public IPs ([10 · EKS](#)). A forgotten Minikube or VM keeps eating your laptop's memory.
- **Leftover state.** A half-removed node keeps old certificates in `/etc/kubernetes`, CNI configuration in `/etc/cni/net.d` and iptables rules. The next `kubeadm init` or `join` then fails in confusing ways.
- **Confusion.** Stale kubeconfig contexts point kubectl at clusters that no longer exist ([troubleshooting 05](#)).

## The four levels

Clean up from the top down: the most specific things first, the machine last.

|         |           |                                                                                     |                                            |
|---------|-----------|-------------------------------------------------------------------------------------|--------------------------------------------|
| Level 1 | Workloads | <code>kubectl delete service/deployment ...</code>                                  | cluster keeps running, ready for more labs |
| Level 2 | Nodes     | <code>drain → delete node → reset the machine</code>                                | cluster keeps running with fewer nodes     |
| Level 3 | Cluster   | <code>kubeadm reset / minikube delete / snap remove / eksctl delete cluster</code>  |                                            |
| Level 4 | Machines  | <code>delete the VMs / check the cloud account is back to its starting state</code> |                                            |

Why the order matters: some level-1 objects create things **outside** the cluster. On EKS, a `Service type: LoadBalancer` makes AWS create a load balancer, and a `PersistentVolumeClaim` makes an EBS volume. If you delete the cluster first, nothing is left to delete them: they stay, keep billing, and the load balancer's network interfaces block the VPC from being deleted.

## kubeadm

### Remove one node (level 2)

⚠️ DESTRUCTIVE COMMAND · evicts every Pod from k8s-worker and removes the node from the cluster.

```
kubectl drain k8s-worker --ignore-daemonsets --delete-emptydir-data --force --timeout=120s
kubectl delete node k8s-worker
```

Then, on the worker itself:

⚠️ DESTRUCTIVE COMMAND · removes all Kubernetes state from this machine.

```
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d
sudo iptables -F && sudo iptables -t nat -F && sudo iptables -t mangle -F && sudo iptables -X
```

What `kubeadm reset` does, and what it does not

| kubeadm reset removes                                                              | kubeadm reset leaves behind (you remove it)                                                                         |
|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| static Pod manifests, certificates and kubeconfigs in <code>/etc/kubernetes</code> | CNI configuration in <code>/etc/cni/net.d</code>                                                                    |
| the local etcd data ( <code>/var/lib/etcd</code> ) on a control plane              | iptables / IPVS rules created by kube-proxy and the CNI                                                             |
| kubelet state ( <code>/var/lib/kubelet</code> ), stops the kubelet's Pods          | <code>\$HOME/.kube/config</code> of every user                                                                      |
|                                                                                    | the installed packages (kubeadm, kubelet, kubectl, containerd)                                                      |
|                                                                                    | CNI network interfaces ( <code>cni0</code> , <code>flannel.1</code> ) until a reboot or <code>ip link delete</code> |

The command prints these reminders itself at the end. The full sequence for both machines is in [kubeadm/cleanup.md](#).

## Remove the machines (level 4)

⚠️ DESTRUCTIVE COMMAND · deletes the virtual machines and their disks.

```

multipass delete k8s-cp k8s-worker
multipass purge

```

## Minikube

⚠️ DESTRUCTIVE COMMAND · deletes the "minikube" cluster: the container, its disk and its kubectl context.

```
minikube delete
```

| Variant                                                   | Removes                                                                                                                          |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>minikube delete</code>                              | the default profile (node container, its volume, kubeconfig context)                                                             |
| <code>minikube delete -p</code><br><code>multinode</code> | one named profile                                                                                                                |
| <code>minikube delete --all</code>                        | every profile                                                                                                                    |
| <code>minikube delete --all --purge</code>                | every profile <b>and</b> <code>~/minikube</code> (cached images, downloaded binaries): the next start downloads everything again |

Details: [minikube/cleanup.md](#).

## MicroK8s

⚠️ DESTRUCTIVE COMMAND · removes MicroK8s, its datastore and every workload on this machine.

```
sudo snap remove microk8s --purge
```

`--purge` skips the automatic snapshot `snappy` would otherwise keep of the snap's data, so nothing is left in `/var/snap/microk8s`. On a multi-node cluster, first remove the other nodes (`microk8s leave` on the node, then `microk8s remove-node <name>` on a remaining one). Then delete the VM as in the [lesson](#).

## Amazon EKS

The order from [eks/cleanup.md](#):

1. `kubectl delete service web` → AWS deletes the load balancer  
`kubectl delete deployment web`
2. `eks/scripts/verify-cleanup.sh | grep 'Load balancers'` wait until it says "gone"
3. `eksctl delete cluster -f eks/cluster.yaml --wait` → CloudFormation deletes node group, then cluster + VPC
4. `eks/scripts/verify-cleanup.sh` checks by name AND tag: cluster, stacks, VPC, NAT, EIPs, security groups, instances, volumes, load balancers, IAM roles
5. `eks/scripts/inventory.sh` the counts must equal the ones you saved before creating the cluster

⚠️ DESTRUCTIVE COMMAND · deletes the EKS cluster `k8s-from-zero`, its nodes and the VPC `eksctl` created for it.

```
eksctl delete cluster -f eks/cluster.yaml --wait
```

`--wait` matters: without it `eksctl` returns while CloudFormation is still deleting, and you might believe it is done. If a stack ends in `DELETE_FAILED`, the CloudFormation console shows which resource blocked it (almost always a leftover load balancer or ENI in the VPC): delete that resource and run the command again.

## Verify, don't assume

Every cleanup page in this course ends with a command that **must fail or come back empty**:

| Cluster  | Proof it is gone                                                                                                                                |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| kubeadm  | <code>kubectl get nodes</code> → connection refused; <code>multipass list</code> no longer lists the VMs                                        |
| Minikube | <code>docker ps -a --filter name=minikube</code> is empty; <code>kubectl config get-contexts minikube</code> fails                              |
| MicroK8s | <code>microk8s status</code> → command not found                                                                                                |
| EKS      | <code>verify-cleanup.sh</code> → Clean: nothing of <code>k8s-from-zero</code> is left. and <code>inventory.sh</code> equals the "before" counts |

## Check yourself

### ▼ Why delete a LoadBalancer Service before deleting an EKS cluster?

The AWS load balancer was created by Kubernetes, not by the `eksctl` CloudFormation stacks. Deleting the cluster first leaves it running (and billing), and its network interfaces stop the VPC from being deleted.

▼ After `kubeadm reset -f`, a new `kubeadm join` fails with network errors. What did reset leave behind?

CNI configuration in `/etc/cni/net.d`, iptables rules and CNI interfaces. Remove them (or reboot) before reusing the machine.

▼ What is the difference between `minikube delete --all` and `minikube delete --all --purge`?

`--purge` also deletes `~/minikube` (cached images and binaries), so the next start has to download everything again.

▼ What does `drain` do before you delete a node?

It marks the node unschedulable and evicts its Pods (respecting `PodDisruptionBudgets`) so they are recreated on other nodes; `DaemonSet` Pods are skipped with `--ignore-daemonsets`.

▼ How do you prove an AWS account is back to its starting state?

Save a read-only inventory before creating anything, then compare it after cleanup, and check by tag and name for every resource type the tool creates (the course's `inventory.sh` and `verify-cleanup.sh`). Also look at the bill the next day.

## 13 · Comparing the installation methods

Time: 15 minutes · Read it after the four installation lessons, then try [lab 05](#).

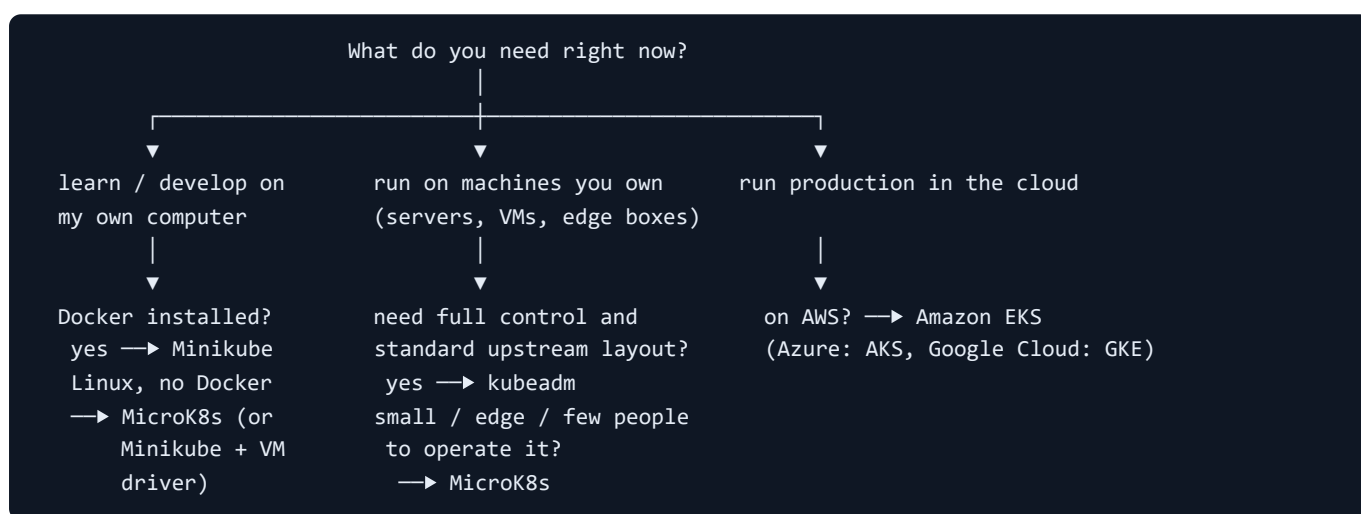
You have now installed Kubernetes four ways. They all give you the **same Kubernetes API**: the same `kubectl` commands and the same YAML work on each. What differs is who runs the control plane, how many machines you need, and what the result is good for.

### Side by side

|                          | kubeadm                                                                                | Minikube                                                                      | MicroK8s                                                                                      | Amazon EKS                                                                            |
|--------------------------|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Purpose</b>           | build a real cluster on machines you control                                           | a local cluster for development and learning                                  | a lightweight cluster for one or a few Linux machines, edge, IoT                              | managed production Kubernetes on AWS                                                  |
| <b>Setup effort</b>      | highest: prepare every node (swap, kernel modules, runtime, packages), init, CNI, join | one command                                                                   | one <code>snap install</code>                                                                 | a config file + one <code>eksctl</code> command; AWS knowledge for networking and IAM |
| <b>Setup time</b>        | the longest by hand (the lesson's two-node build is the longest lesson)                | a few minutes after the first download                                        | a few minutes                                                                                 | about 15 minutes for AWS to create it (14.5 min in the lesson's run)                  |
| <b>Nodes</b>             | any number of machines                                                                 | 1 (or several containers on one computer)                                     | 1 to many machines                                                                            | managed node groups, Auto Mode, Fargate                                               |
| <b>High availability</b> | yes, if you build it: 3 control-plane nodes + a load balancer for the API, etcd quorum | no                                                                            | yes, automatic from 3 nodes                                                                   | yes, built in: the control plane runs in 3 availability zones                         |
| <b>Default CNI</b>       | none, you choose (Flannel in the lesson)                                               | built in (kindnet for multi-node; bridge for one node)                        | Calico                                                                                        | Amazon VPC CNI (Pods get VPC IPs)                                                     |
| <b>Datastore</b>         | etcd (static Pod on the control plane)                                                 | etcd inside the node                                                          | dqlite                                                                                        | etcd run by AWS (invisible to you)                                                    |
| <b>Container runtime</b> | your choice of CRI runtime; containerd in the lesson                                   | containerd inside the node container                                          | containerd bundled in the snap                                                                | containerd on the EKS AMI (AL2023)                                                    |
| <b>Upgrades</b>          | you: <code>kubeadm upgrade</code> node by node, plus packages                          | <code>minikube start --kubernetes-version=...</code> or <code>recreate</code> | <code>snap refresh</code> (patches automatic within a track; you change the track for minors) | control plane: one click/command; nodes: managed rolling update                       |

|                       | kubeadm                                                         | Minikube                           | MicroK8s                                        | Amazon EKS                                                                |
|-----------------------|-----------------------------------------------------------------|------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------|
| <b>Cost</b>           | your hardware or VMs                                            | free (your laptop)                 | free (your machines)                            | per-hour control plane + EC2 + NAT + load balancers (lab ≈ \$0.25–0.30/h) |
| <b>Production use</b> | yes: on-premises and bare metal, the base of many distributions | no                                 | yes for edge, small and appliance-like clusters | yes, very common                                                          |
| <b>Learning value</b> | highest: you see and configure every component                  | fast feedback for kubectl and YAML | fast, shows a different packaging model         | how real cloud teams run Kubernetes: VPC, IAM, load balancers, cost       |

## Which one should I use?



And for **understanding how Kubernetes works** (exams, interviews, debugging): build it once with kubeadm.

## Real-world scenarios

| Scenario                                                              | Good choice                                                     | Why                                                                                |
|-----------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------|
| A developer tests Helm charts and manifests on a laptop               | Minikube                                                        | one command, deleted in seconds, runs next to Docker                               |
| A CI pipeline needs a throwaway cluster for every pull request        | Minikube (or kind) on the CI runner; MicroK8s on a Linux runner | starts in minutes, no cloud cost, removed with the runner                          |
| Shops or factory sites each run a small cluster on two or three boxes | MicroK8s                                                        | snap updates, small footprint, HA from three nodes                                 |
| A company runs Kubernetes in its own data center                      | kubeadm (or a distribution built on it)                         | standard upstream layout, full control over network, storage and upgrades          |
| A team runs customer-facing services on AWS                           | Amazon EKS                                                      | no control plane to operate, integrates with VPC, IAM, load balancers, autoscaling |
| Preparing for the CKA exam                                            | kubeadm                                                         | the exam asks you to install, upgrade, back up and troubleshoot kubeadm clusters   |

## What stays the same everywhere

- The API, `kubectl`, and the YAML in [examples/nginx](#).
- The [verification checklist](#): the same eight checks pass on every cluster.
- The troubleshooting method: `get` → `describe` → `logs` → node services.

## Check yourself

### ▼ Which method gives you a highly-available control plane without any extra work?

Amazon EKS: AWS runs the control plane across three availability zones. MicroK8s also becomes HA automatically, but only after you add three nodes.

### ▼ You need a cluster in an air-gapped factory on three small industrial PCs. Which method fits, and why?

MicroK8s: small footprint, one package per node, automatic HA with three nodes, and snaps can be installed offline. kubeadm also works but needs more operations effort.

### ▼ Why is kubeadm the best way to learn, even if you will use EKS at work?

You set up every component yourself (runtime, kubelet, certificates, CNI, join), so you understand what EKS hides and can debug the parts you still own (nodes, CNI, workloads).

### ▼ Will a Deployment YAML written on Minikube work on EKS?

Yes, the API is the same. What usually changes is how it is exposed (LoadBalancer vs NodePort), storage classes, and resource sizing.

Back to the roadmap: [../README.md](#)

# Troubleshooting labs

Every lab **breaks something on purpose**, shows you the symptom exactly as you would meet it at work, and walks through the investigation in the same order every time:

Problem → Symptoms → Initial investigation → Commands → Output interpretation → Root cause → Fix → Verification → Lesson learned

All outputs shown in the labs are real: they were produced by running the labs end to end on fresh virtual machines.

| #  | Lab                                    | Cluster          | What breaks                                              | Main tools                                                                               |
|----|----------------------------------------|------------------|----------------------------------------------------------|------------------------------------------------------------------------------------------|
| 01 | <a href="#">Node is NotReady</a>       | kubeadm          | the kubelet stops on the worker                          | <code>kubectl describe node</code> ,<br><code>systemctl</code> , <code>journalctl</code> |
| 02 | <a href="#">Pod networking</a>         | kubeadm          | IP forwarding is switched off on a node                  | <code>curl</code> , <code>ip route</code> , <code>sysctl</code>                          |
| 03 | <a href="#">kubeadm init fails</a>     | new VM           | containerd's CRI plugin is disabled                      | <code>crictl</code> , <code>config.toml</code>                                           |
| 04 | <a href="#">Worker cannot join</a>     | kubeadm + new VM | expired join token, blocked port 6443                    | <code>nc</code> , <code>/version</code> , <code>kubeadm token list</code>                |
| 05 | <a href="#">kubectl cannot connect</a> | kubeadm          | missing kubeconfig, wrong server address                 | <code>kubectl config</code>                                                              |
| 06 | <a href="#">Core Pods not running</a>  | kubeadm          | a typo in the CoreDNS configuration                      | <code>describe</code> , <code>logs --previous</code> ,<br><code>rollout</code>           |
| 07 | <a href="#">Container runtime down</a> | kubeadm          | containerd stops on the worker                           | <code>crictl</code> , <code>journalctl -u kubelet</code>                                 |
| 08 | <a href="#">Insufficient resources</a> | Minikube         | too little memory for a cluster, a Pod that does not fit | <code>describe pod</code> events, node allocatable                                       |

Labs 01–07 run on the two-node cluster of the [kubeadm lesson](#), in this order (03 creates the VM that 04 uses and removes). Lab 08 runs on the [Minikube](#) cluster.

## The general method

- |                                        |                                                                                                          |
|----------------------------------------|----------------------------------------------------------------------------------------------------------|
| 1. What exactly is the symptom?        | <code>kubectl get nodes / pods -A</code> , read the STATUS column                                        |
| 2. What does Kubernetes know about it? | <code>kubectl describe &lt;object&gt;</code> → Conditions, Events                                        |
| 3. What does the component say?        | <code>kubectl logs (--previous)</code> / <code>journalctl -u kubelet</code> / <code>-u containerd</code> |
| 4. Which layer is it?                  | network → runtime → kubelet → API server → workload                                                      |
| 5. Change one thing, then verify fixed | the same command that showed the symptom must now show it fixed                                          |

# Capstone · prove it

After all ten levels of the [roadmap](#). Time: 60–90 minutes. Part A without notes, Part B with a terminal.

## Part A · 15 questions

Answer in your own words first. Then open the answer and compare.

**1. A colleague says "Kubernetes runs my containers". Which component actually starts a container on a node, and which components are involved between `kubectl apply` and the running container?**

### ▼ Answer

`kubectl` sends the object to the **API server**, which validates it and stores it in **etcd**. The **controller manager** (Deployment and ReplicaSet controllers) creates Pod objects; the **scheduler** assigns each Pod to a node; the **kubelet** on that node sees the Pod and asks the **container runtime** (containerd, through the CRI) to pull the image and start the container, which `runc` finally creates. See [docs/02](#).

**2. Why must swap be handled, and why are `br_netfilter` and `net.ipv4.ip_forward` needed?**

### ▼ Answer

The kubelet's resource accounting assumes memory limits are real; by default it refuses to start with swap on (newer versions support swap only when explicitly configured). `br_netfilter` + `bridge-nf-call-iptables` make traffic that crosses a Linux bridge (Pod to Pod on a node) go through iptables, where kube-proxy's Service rules live. `ip_forward` lets the node route packets between Pods, other nodes and the outside: without it cross-node Pod traffic dies, which is exactly [troubleshooting 02](#).

**3. What is the CRI, and why did "Docker support" disappear from Kubernetes in 1.24?**

### ▼ Answer

The Container Runtime Interface is the gRPC API the kubelet uses to talk to a runtime. Docker Engine never implemented it; Kubernetes carried an adapter (dockershim), which was removed in 1.24. Runtimes that implement CRI (containerd, CRI-O) are used directly. Images built with Docker still run everywhere: they are standard OCI images.

**4. containerd is `active`, but `kubeadm init` fails its preflight with a runtime error. What do you check?**

### ▼ Answer

Whether the **CRI plugin** answers: `sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock version`. The `containerd.io` package config disables it (`disabled_plugins = ["cri"]`); regenerate the config with `containerd config default`, set `SystemdCgroup = true`, restart. [Troubleshooting 03](#).

**5. Why is the control plane `NotReady` right after `kubeadm init`, and what fixes it?**

### ▼ Answer

No CNI plugin is installed yet: the kubelet reports `NetworkReady=false reason:NetworkPluginNotReady ... cni plugin not initialized`. Installing a CNI (Flannel in the lesson, matching `--pod-network-cidr=10.244.0.0/16`) makes it Ready.

## 6. A worker's join hangs at discovery. Name three causes and the command that tells them apart.

### ▼ Answer

Port 6443 blocked (firewall / security group) → `nc -zv <cp-ip> 6443` fails; wrong API server address → same; expired or wrong token → network fine, `curl -k https://<cp-ip>:6443/version` answers, but the token is missing from `kubeadm token list`. A wrong CA hash is reported explicitly by the join. [Troubleshooting 04](#).

## 7. `kubectl get nodes` says `The connection to the server localhost:8080 was refused`. What does that tell you?

### ▼ Answer

`kubectl` found **no kubeconfig** at all and fell back to its default. Check `$KUBECONFIG`, `~/.kube/config`, and `kubectl config current-context`. [Troubleshooting 05](#).

## 8. A node is `NotReady`. How do you tell "the kubelet is dead" from "the runtime is dead"?

### ▼ Answer

`kubectl describe node` → Conditions. Kubelet gone: Kubelet stopped posting node status (no fresh heartbeats, node gets the `unreachable` taint). Runtime gone: the kubelet still reports, with a container-runtime message; on the node `systemctl is-active containerd` is inactive and `crictl ps` cannot connect. [01](#), [07](#).

## 9. CoreDNS Pods are in `CrashLoopBackOff`. What is your sequence of commands?

### ▼ Answer

`kubectl get pods -n kube-system -l k8s-app=kube-dns` → `kubectl describe pod` (exit code, back-off events) → `kubectl logs --previous` (the crash reason, e.g. an unknown Corefile directive) → fix the ConfigMap (from a backup) → `kubectl rollout restart` + `rollout status` → test DNS from a Pod with `nslookup`. [Troubleshooting 06](#).

## 10. A Pod stays `Pending`. Where is the reason, and what does `Insufficient cpu` mean exactly?

### ▼ Answer

In the Pod's **Events** (`kubectl describe pod`), written by the scheduler. `Insufficient cpu`: on no node does allocatable CPU minus the **requests** of Pods already there leave room for this Pod's request. Real usage does not count. Fix the request or add capacity. [Troubleshooting 08](#).

## 11. What is the difference between Minikube, MicroK8s and kubeadm in where the control plane runs?

### ▼ Answer

kubeadm: static Pods (API server, etcd, scheduler, controller manager) managed by the kubelet on the control-plane machine. Minikube: a whole node (with kubeadm inside) running as a Docker container (or VM) on your computer. MicroK8s: all components in one process ( `kubelite` ) installed by a snap, with `dqlite` instead of etcd.

## 12. On EKS, what do you manage and what does AWS manage?

### ▼ Answer

AWS: the control plane (API servers, etcd) across availability zones, its scaling, patching, backups and certificates. You: the VPC, node groups (instance type, size, AMI updates), add-on versions, IAM (cluster and node roles, access entries), workloads, and cluster version upgrades (which you trigger). [docs/10](#).

## 13. Why must LoadBalancer Services be deleted before `eksctl delete cluster` ?

### ▼ Answer

The load balancer was created by Kubernetes (the cloud controller), not by eksctl's CloudFormation stacks. If the cluster disappears first, nothing deletes the load balancer: it keeps costing money, and its network interfaces and security group block the deletion of the VPC. [eks/cleanup.md](#).

## 14. What does `kubeadm reset` not clean up?

### ▼ Answer

The CNI configuration in `/etc/cni/net.d`, iptables/IPVS rules, and kubeconfig files in users' home directories (e.g. `~/.kube/config`). It also does not uninstall packages or containerd. [kubeadm/cleanup.md](#), [docs/12](#).

## 15. Your Pods on EKS have IPs like `10.50.159.242`, on kubeadm `10.244.1.5`. Why the difference, and what is one consequence?

### ▼ Answer

The Amazon VPC CNI gives Pods real VPC addresses from the subnets; Flannel gives them addresses from its own overlay range ( `10.244.0.0/16` ) and encapsulates traffic in VXLAN. Consequences: on EKS, Pods are directly reachable inside the VPC and visible to security groups and flow logs, but they consume subnet IPs and the number of Pods per node is limited by the instance's network interfaces. [docs/06](#).

## Part B · Scenario: the Monday morning cluster

You join a team on Monday. Their **kubeadm** test cluster ( `k8s-cp` + `k8s-worker` ) "stopped working over the weekend". Rebuild that situation from the lessons, then fix it as if you did not know what was done.

### Set it up (on your computer, with the kubeadm cluster from the lesson running)

Ask a friend to run these, or run them and then wait a day before you start:

```
# three faults at once
multipass exec k8s-worker -- sudo systemctl stop containerd
multipass exec k8s-worker -- sudo sysctl -w net.ipv4.ip_forward=0
multipass exec k8s-cp -- bash -c 'cp ~/.kube/config ~/.kube/config.bak && sed -i "s#server:
https://.*:6443#server: https://10.255.255.1:6443#" ~/.kube/config'
```

## Your task

1. Find **every** fault, starting from what a user would see ( `kubectl get nodes` on the control plane).
2. For each fault, write down: symptom → command that proves the cause → root cause → fix → verification.
3. Restore the cluster to a fully verified state with the [verification checklist](#): nodes Ready, system Pods Running, DNS works, the test app answers through its NodePort **and** a Pod on one node can reach a Pod on the other.
4. Write a short incident note (5–10 lines): what broke, impact, how you found it, how you fixed it, how to prevent it.

## Hints

- Fix the layer that hides the others first: you cannot investigate nodes while `kubectl` cannot reach the API.
- After the node is `Ready`, everything may *look* fine. Test traffic across nodes.

### ▼ Solution outline

1. `kubectl get nodes` → i/o timeout to 10.255.255.1: wrong server in `~/.kube/config` ( `kubectl config view --minify` ). Fix: `restore config.bak` or copy `/etc/kubernetes/admin.conf` again.
2. `kubectl get nodes` → `k8s-worker NotReady`; describe node shows a container-runtime message; on the worker `systemctl is-active containerd` → inactive. Fix: `sudo systemctl start containerd`; wait for Ready.
3. Checklist: NodePort on the worker answers locally, but `curl` from the control plane to a Pod IP on the worker times out; on the worker `sysctl net.ipv4.ip_forward` → 0. Fix: `sudo sysctl --system` (the value in `/etc/sysctl.d/k8s.conf` is 1), then re-run the full checklist.

Prevention: configuration management for kubeconfig and sysctls, node monitoring (NotReady alerts), and an automated smoke test that includes cross-node traffic.

# Glossary

---

Every term used in this course, in plain words. The link points to where it is explained or used.

| Term                                     | Meaning                                                                                                                                                                    |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Access entry</b>                      | An EKS object that gives an AWS identity (user or role) permissions inside the Kubernetes cluster. AWS permissions alone are not enough. <a href="#">docs/10</a>           |
| <b>Add-on</b>                            | An optional, pre-packaged component (DNS, metrics, ingress, storage). Minikube, MicroK8s and EKS all have their own add-on mechanism. <a href="#">docs/08</a>              |
| <b>admin.conf</b>                        | The kubeconfig kubeadm writes with cluster-admin rights. Treat it like a root password. <a href="#">docs/05</a>                                                            |
| <b>Allocatable</b>                       | The CPU and memory of a node that Pods may request, after reserving some for the system. <a href="#">troubleshooting 08</a>                                                |
| <b>Amazon EKS</b>                        | AWS's managed Kubernetes: AWS runs the control plane, you run the nodes and workloads. <a href="#">eks/README.md</a>                                                       |
| <b>API server (kube-apiserver)</b>       | The front door of the cluster. Every component and every kubectl command talks to it, and only it talks to etcd. <a href="#">docs/02</a>                                   |
| <b>Availability zone (AZ)</b>            | One or more separate data centres inside an AWS region. Spreading nodes over AZs survives the loss of one. <a href="#">docs/10</a>                                         |
| <b>aws-node</b>                          | The DaemonSet of the Amazon VPC CNI on EKS nodes. <a href="#">labs/04</a>                                                                                                  |
| <b>Bootstrap token</b>                   | A short-lived token that lets a new node authenticate when it joins with <code>kubeadm join</code> . Expires after 24 hours by default. <a href="#">troubleshooting 04</a> |
| <b>br_netfilter</b>                      | A kernel module that makes bridged traffic visible to iptables, where Service rules live. <a href="#">docs/03</a>                                                          |
| <b>CA certificate hash</b>               | A fingerprint of the cluster's certificate authority. The joining node uses it to verify it talks to the real control plane. <a href="#">docs/05</a>                       |
| <b>Calico</b>                            | A CNI plugin with network policy support; the default CNI of MicroK8s. <a href="#">docs/06</a>                                                                             |
| <b>cgroup / cgroup driver</b>            | The Linux feature that limits and accounts for CPU and memory. The kubelet and the runtime must use the same driver (systemd here). <a href="#">docs/04</a>                |
| <b>Channel (snap)</b>                    | A version track of a snap package, like <code>1.36/stable</code> for MicroK8s. <a href="#">docs/09</a>                                                                     |
| <b>CloudFormation stack</b>              | A group of AWS resources created and deleted together from a template. eksctl creates one for the cluster and one per node group. <a href="#">eks/README.md</a>            |
| <b>Cluster</b>                           | A set of nodes managed by one control plane. <a href="#">docs/01</a>                                                                                                       |
| <b>CNI (Container Network Interface)</b> | The standard for network plugins that give Pods their network. Without one, nodes stay <code>NotReady</code> . <a href="#">docs/06</a>                                     |
| <b>ConfigMap</b>                         | A Kubernetes object holding configuration data, such as CoreDNS's Corefile. <a href="#">troubleshooting 06</a>                                                             |
| <b>containerd</b>                        | The container runtime used in this course. The kubelet talks to it through the CRI. <a href="#">docs/04</a>                                                                |

| Term                                                | Meaning                                                                                                                                                              |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Context</b>                                      | A named entry in a kubeconfig: which cluster, which user, which namespace. <a href="#">troubleshooting 05</a>                                                        |
| <b>Control plane</b>                                | The components that make decisions for the cluster: API server, etcd, scheduler, controller manager. <a href="#">docs/02</a>                                         |
| <b>Controller manager (kube-controller-manager)</b> | Runs the control loops that make actual state match desired state (Deployments, ReplicaSets, nodes, ...). <a href="#">docs/02</a>                                    |
| <b>CoreDNS</b>                                      | The cluster DNS server that lets Pods find Services by name. <a href="#">troubleshooting 06</a>                                                                      |
| <b>CrashLoopBackOff</b>                             | A Pod status: the container keeps exiting and Kubernetes restarts it with growing delays. The cause is in the logs. <a href="#">troubleshooting 06</a>               |
| <b>CRI (Container Runtime Interface)</b>            | The API the kubelet uses to talk to a container runtime. <a href="#">docs/04</a>                                                                                     |
| <b>crictl</b>                                       | The command-line client for the CRI; the way to check a runtime as the kubelet sees it. <a href="#">troubleshooting 03</a>                                           |
| <b>DaemonSet</b>                                    | A workload that runs one Pod on every (matching) node, used by CNI plugins and kube-proxy. <a href="#">docs/06</a>                                                   |
| <b>Deployment</b>                                   | A workload that keeps a number of identical Pods running and replaces them during updates. <a href="#">examples/nginx</a>                                            |
| <b>Desired state</b>                                | What you declared in Kubernetes objects. Controllers keep reality matching it. <a href="#">docs/01</a>                                                               |
| <b>dqlite</b>                                       | The distributed SQLite datastore MicroK8s uses instead of etcd. <a href="#">docs/09</a>                                                                              |
| <b>Drain</b>                                        | Evicting all Pods from a node before maintenance or removal ( <code>kubect1 drain</code> ). <a href="#">kubeadm/cleanup.md</a>                                       |
| <b>Driver (Minikube)</b>                            | What Minikube runs its node in: a Docker container, a VM (Hyper-V, KVM, QEMU) or Podman. <a href="#">docs/08</a>                                                     |
| <b>eksctl</b>                                       | The official command-line tool that creates and deletes EKS clusters from a YAML file. <a href="#">eks/README.md</a>                                                 |
| <b>Endpoint (Service)</b>                           | The list of Pod IPs a Service currently sends traffic to. Empty means no matching Ready Pods. <a href="#">examples/nginx</a>                                         |
| <b>etcd</b>                                         | The key-value store holding the whole cluster state. Back it up on self-managed clusters. <a href="#">docs/02</a>                                                    |
| <b>Events</b>                                       | Short records of what happened to an object (scheduled, pulled, failed, ...). Shown at the end of <code>kubect1 describe</code> . <a href="#">troubleshooting 08</a> |
| <b>Flannel</b>                                      | A simple CNI plugin that connects nodes with a VXLAN overlay; used in the kubeadm lesson. <a href="#">docs/06</a>                                                    |
| <b>Ingress / ingress controller</b>                 | An Ingress is an HTTP routing rule (host and path to a Service); the controller is the program that carries it out. <a href="#">labs/03</a>                          |
| <b>ip_forward</b>                                   | The Linux setting that lets a machine route packets. Without it, Pod traffic between nodes stops. <a href="#">troubleshooting 02</a>                                 |

| Term                               | Meaning                                                                                                                                                                                      |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>kubeadm</b>                     | The official tool that turns prepared Linux machines into a cluster ( <code>init</code> , <code>join</code> , <code>reset</code> , <code>upgrade</code> ). <a href="#">kubeadm/README.md</a> |
| <b>kubeconfig</b>                  | The file that tells kubectl where the API server is and how to authenticate. Default <code>~/.kube/config</code> . <a href="#">troubleshooting 05</a>                                        |
| <b>kubectl</b>                     | The command-line client for the Kubernetes API. <a href="#">docs/02</a>                                                                                                                      |
| <b>kubelet</b>                     | The agent on every node that starts Pods through the runtime and reports their status. <a href="#">docs/02</a>                                                                               |
| <b>kubelite</b>                    | MicroK8s's single process that contains the API server, scheduler, controller manager, kubelet and proxy. <a href="#">docs/09</a>                                                            |
| <b>kube-proxy</b>                  | Runs on every node and programs the rules that send Service traffic to Pods. <a href="#">docs/02</a>                                                                                         |
| <b>Label / selector</b>            | Key-value tags on objects, and the queries that pick objects by them. Services find their Pods this way. <a href="#">examples/nginx</a>                                                      |
| <b>Liveness / readiness probe</b>  | Health checks: liveness restarts a stuck container, readiness decides whether a Pod gets traffic. <a href="#">examples/nginx</a>                                                             |
| <b>LoadBalancer (Service type)</b> | A NodePort Service plus a cloud load balancer in front of it. <a href="#">eks/README.md</a>                                                                                                  |
| <b>Managed node group</b>          | EKS-managed EC2 workers in an Auto Scaling group that join the cluster on their own. <a href="#">docs/10</a>                                                                                 |
| <b>Minikube</b>                    | A tool that runs a complete local cluster, usually in one Docker container. <a href="#">minikube/README.md</a>                                                                               |
| <b>MicroK8s</b>                    | Kubernetes packaged as one snap, for single machines and edge devices. <a href="#">microk8s/README.md</a>                                                                                    |
| <b>Namespace</b>                   | A named section of a cluster that groups objects ( <code>kube-system</code> , <code>default</code> , ...). <a href="#">docs/02</a>                                                           |
| <b>NAT gateway</b>                 | An AWS service that lets machines in private subnets reach the internet. Billed per hour, a classic forgotten cost. <a href="#">eks/README.md</a>                                            |
| <b>Node</b>                        | A machine (VM, server, container in Minikube) that runs Pods. <a href="#">docs/02</a>                                                                                                        |
| <b>NodePort</b>                    | A Service type that opens the same port (30000–32767) on every node. <a href="#">kubeadm/README.md</a>                                                                                       |
| <b>NotReady</b>                    | A node status: the kubelet reports a problem (no CNI, runtime down) or stopped reporting at all. <a href="#">troubleshooting 01</a>                                                          |
| <b>overlay (module)</b>            | The filesystem driver containerd uses to stack image layers. <a href="#">docs/03</a>                                                                                                         |
| <b>Pending</b>                     | A Pod status: not yet placed on a node, or its containers not created yet. The Events say why. <a href="#">troubleshooting 08</a>                                                            |
| <b>Pod</b>                         | The smallest unit Kubernetes runs: one or more containers sharing a network address. <a href="#">docs/01</a>                                                                                 |
| <b>Pod network CIDR</b>            | The address range for Pod IPs, <code>10.244.0.0/16</code> for Flannel in the kubeadm lesson. <a href="#">docs/06</a>                                                                         |

| Term                              | Meaning                                                                                                                                                |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Preflight checks</b>           | kubeadm's checks before it changes anything (CPUs, memory, swap, ports, runtime). <a href="#">troubleshooting 03</a>                                   |
| <b>Profile (Minikube)</b>         | A separately named Minikube cluster with its own kubectl context. <a href="#">labs/02</a>                                                              |
| <b>Requests and limits</b>        | Requests are what the scheduler reserves for a container; limits are the hard maximum. <a href="#">troubleshooting 08</a>                              |
| <b>Rolling update</b>             | Replacing Pods gradually, keeping the old ones until the new ones are Ready. <a href="#">troubleshooting 06</a>                                        |
| <b>runc</b>                       | The low-level tool containerd uses to actually create containers. <a href="#">docs/04</a>                                                              |
| <b>Scheduler (kube-scheduler)</b> | Chooses a node for each new Pod, based on requests, taints, affinity and more. <a href="#">docs/02</a>                                                 |
| <b>Service</b>                    | A stable virtual address and DNS name in front of a changing set of Pods. <a href="#">docs/02</a>                                                      |
| <b>Snap</b>                       | Ubuntu's package format with versioned channels and automatic refreshes; MicroK8s ships as one. <a href="#">docs/09</a>                                |
| <b>Static Pod</b>                 | A Pod the kubelet starts from a file on disk; kubeadm runs the control plane this way. <a href="#">docs/05</a>                                         |
| <b>Swap</b>                       | Disk used as extra memory. The kubelet refuses to run with it by default. <a href="#">docs/03</a>                                                      |
| <b>Taint / toleration</b>         | A taint keeps Pods off a node unless they carry a matching toleration. The control plane is tainted. <a href="#">kubeadm/README.md</a>                 |
| <b>Version skew</b>               | The supported difference between component versions, for example kubectl one minor version newer or older than the API server. <a href="#">labs/02</a> |
| <b>VPC / subnet</b>               | Your private network in AWS, divided into subnets per availability zone. <a href="#">docs/10</a>                                                       |
| <b>VPC CNI (Amazon)</b>           | The EKS network plugin that gives Pods real IP addresses from the VPC's subnets. <a href="#">docs/06</a>                                               |
| <b>VXLAN</b>                      | A way to tunnel Layer 2 traffic over UDP; Flannel uses it to connect Pod networks across nodes. <a href="#">docs/06</a>                                |

# Interview questions

---

25 questions that come up in interviews for junior and intermediate DevOps, platform and cloud roles. Answer each one out loud first, then open the model answer. Each answer links to the lesson where you did it yourself: in an interview, "I did this in a lab, and here is what I saw" beats any memorised definition.

## Fundamentals



### 1. What problem does Kubernetes solve?

Running containers on one machine is easy; running many containers across many machines reliably is not. Kubernetes places containers on machines, restarts them when they fail, replaces machines that disappear, gives them stable addresses and names, and rolls out new versions gradually. You declare the desired state, and controllers keep reality matching it. ([docs/01](#))



### 1. Describe the components of a Kubernetes cluster.

The control plane has the API server (the only entry point), etcd (the state), the scheduler (chooses nodes for Pods) and the controller manager (the reconciliation loops), plus a cloud controller manager on clouds. Every node runs the kubelet (starts and watches Pods), kube-proxy (Service rules) and a container runtime such as containerd. Add-ons such as CoreDNS and a CNI plugin complete it. ([docs/02](#))



### 1. What happens when you run `kubectl apply -f deployment.yaml` ?

kubectl sends the object to the API server, which authenticates, authorises, validates and stores it in etcd. The Deployment controller creates a ReplicaSet, which creates Pod objects. The scheduler assigns each Pod to a node. The kubelet on that node sees the Pod, asks containerd through the CRI to pull the image and start the containers, and reports the status back. ([docs/02](#))



### 1. What is etcd, and why is it critical?

etcd is a distributed key-value store that holds every object in the cluster. If it is lost without a backup, the cluster's entire state is lost, even if all the machines are fine. On self-managed clusters you back it up regularly and test restores; on managed services such as EKS, the provider does it. ([docs/05](#))

## Installation



### 1. What do you have to prepare on a Linux machine before running kubeadm?

Unique hostname, MAC and product\_uuid; swap off; the `overlay` and `br_netfilter` kernel modules; IP forwarding and bridged traffic through iptables; synchronised time; open ports (6443, 10250, etcd and the NodePort range); a container runtime with the CRI enabled and the systemd cgroup driver; and kubeadm, kubelet and kubectl of the same minor version, held against accidental upgrades. ([kubeadm Steps 1–4](#))



### 1. Why was "Docker support" removed from Kubernetes, and does that mean Docker images no longer work?

Docker Engine does not implement the CRI, so Kubernetes maintained an adapter called dockershim, which was removed in 1.24. Kubernetes now talks directly to CRI runtimes such as containerd or CRI-O. Images built with Docker are standard OCI images and run unchanged. ([docs/04](#))



### 1. Walk me through what `kubeadm init` does.

It runs preflight checks, creates a certificate authority and the component certificates, writes kubeconfig files, and writes static Pod manifests for the API server, etcd, scheduler and controller manager, which the kubelet then starts. It waits for the API server, stores its configuration in the cluster, taints the control-plane node, creates a bootstrap token, and installs CoreDNS and kube-proxy. Its output ends with the next steps: kubeconfig, a CNI plugin, and the join command. ([docs/05](#))



### 1. How does a new node join a kubeadm cluster securely?

It needs the API server address, a bootstrap token and the CA certificate hash. The token proves to the cluster that the node may join; the hash lets the node verify it is talking to the real control plane. The kubelet then gets its own client certificate through TLS bootstrapping. `kubeadm token create --print-join-command` produces a fresh command. ([kubeadm Step 7](#))



### 1. When would you use Minikube, MicroK8s, kubeadm or a managed service?

Minikube for development and CI: a disposable cluster in one command. MicroK8s for single machines, edge and IoT fleets: one package, small footprint, optional HA. kubeadm when you must run Kubernetes on your own machines with full control, and to learn the internals. A managed service such as EKS for production in the cloud, when you want the provider to run a highly available control plane. ([docs/13](#), [labs/05](#))



### 1. How does a production kubeadm cluster differ from the lab?

Three or more control-plane nodes behind a load balancer ( `--control-plane-endpoint` ), possibly external etcd, several workers in different failure domains, regular etcd backups, certificate renewal, a tested upgrade process, hardening, monitoring and persistent storage. The lab has one of each to show the mechanics. ([docs/05](#) · [Production is different](#))

## Networking



### 1. Explain the Kubernetes network model.

Every Pod gets its own IP address; every Pod can reach every other Pod on any node without NAT; agents on a node can reach all Pods on that node. Kubernetes defines the model but does not implement it: a CNI plugin does. ([docs/06](#))



### 1. Why is a freshly initialised node `NotReady` ?

No CNI plugin is installed yet. The kubelet reports `NetworkReady=false reason:NetworkPluginNotReady ... cni plugin not initialized`, and CoreDNS stays Pending for the same reason. Installing a CNI matching the Pod CIDR, Flannel in the lab, makes the node Ready. ([kubeadm Steps 5–6](#))



### 1. Compare Flannel with the Amazon VPC CNI.

Flannel builds an overlay: Pods get addresses from a separate range ( `10.244.0.0/16` ) and traffic between nodes is tunnelled with VXLAN. The VPC CNI gives Pods real IP addresses from the VPC subnets, so they are directly routable in the VPC and visible to security groups and flow logs, at the price of consuming subnet IPs and a per-instance Pod limit. ([docs/06](#), [labs/04](#))



### 1. What is the difference between ClusterIP, NodePort, LoadBalancer and Ingress?

ClusterIP is a virtual address reachable only inside the cluster. NodePort opens one port on every node. LoadBalancer adds a cloud load balancer in front of a NodePort. Ingress is an HTTP routing rule (host and path to a Service) carried out by an ingress controller, so many applications can share one entry point on port 80/443. ([labs/03](#), [eks/README.md](#))

## Troubleshooting



### 1. A node is `NotReady` . What do you do?

`kubectl describe node` and read the `Ready` condition's message. "Kubelet stopped posting node status" means the kubelet is down or unreachable: check `systemctl status kubelet` and `journalctl -u kubelet` on the node. A runtime message means containerd is down: `systemctl is-active containerd`, `crictrl ps`. A network-plugin message points at the CNI. Fix one thing, then verify the node turns `Ready`. ([troubleshooting 01, 07](#))



#### 1. A Pod is in `CrashLoopBackOff`. How do you find the cause?

`kubectl describe pod` shows exit codes, restart counts and events; `kubectl logs --previous` shows what the last crashed instance printed, which usually names the problem. `CrashLoopBackOff` is a symptom: the container keeps exiting. Common causes are bad configuration, missing secrets or files, failing dependencies and wrong commands. ([troubleshooting 06](#))



#### 1. A Pod stays `Pending`. What could it be?

Look at its Events. `Insufficient cpu/memory`: the requests don't fit on any node. `untolerated taint`: the only nodes with room are tainted. Node selectors or affinity that match no node, or a volume that cannot be bound, are the other usual causes. Pending Pods have no logs, because no container was started. ([troubleshooting 08](#))



#### 1. `kubeadm join` hangs. How do you debug it?

From the bottom up. Network: can the node reach the API server on 6443 ( `nc -zv <ip> 6443` )? API server: does `curl -k https://<ip>:6443/version` answer? Credentials: is the token still in `kubeadm token list` (they expire), and does the CA hash match? Usually the answer is a firewall or security group, or a stale join command. ([troubleshooting 04](#))



#### 1. Everything is green, but services on different nodes cannot talk. Where do you look?

At the Pod network between nodes. Test a request from one node to a Pod IP on another. Then check IP forwarding on the nodes, the CNI's interfaces and routes, and firewalls or security groups for the overlay traffic (UDP 8472 for Flannel VXLAN). Health checks that only test locally will not catch this. ([troubleshooting 02](#))

## Managed Kubernetes / EKS



#### 1. What does AWS manage in EKS, and what do you manage?

AWS runs the control plane (API servers, etcd, controllers) across availability zones, including its availability, patching, backups and the control-plane part of upgrades you trigger. You manage the VPC, the worker nodes or node groups, IAM access, add-on versions, cluster upgrades, your workloads and their security, and the cost. ([eks/README.md](#), [docs/10](#))



### 1. How do users authenticate to EKS, and why might someone with AWS admin rights see "Unauthorized"?

kubectl calls `aws eks get-token` with the user's AWS identity; EKS maps that identity to Kubernetes permissions through access entries (or the older `aws-auth` ConfigMap). An identity without an access entry is refused by Kubernetes even if its AWS policies allow everything. The console behaves the same way: it reads Pods and nodes through the Kubernetes API. ([eks Step 8](#))



### 1. What costs money in a small EKS lab, and how do you make sure nothing is left?

The control plane per hour, the EC2 workers, the NAT gateway, public IPv4 addresses, load balancers created by Services, and EBS volumes. Tag everything, delete LoadBalancer Services before the cluster, delete with the same tool that created it ( `eksctl delete cluster` ), then verify by name, by tag and by real state, and compare an inventory of the region with the one taken before. ([eks/cleanup.md](#))

## Operations



### 1. How do you safely remove a node from a cluster?

`kubectl drain` the node (evicts Pods, respecting disruption budgets, and marks it unschedulable), `kubectl delete node`, then on the machine `kubeadm reset` and clean the CNI configuration and iptables rules it leaves behind, and only then remove the machine. ([kubeadm/cleanup.md](#), [labs/01](#))



### 1. Why do you hold the Kubernetes packages with `apt-mark hold` ?

So that a routine `apt upgrade` never upgrades kubeadm, kubelet or kubectl behind your back. Cluster upgrades have their own order: control plane first with `kubeadm upgrade`, then the kubelets node by node, one minor version at a time, draining each node. ([kubeadm Step 4](#))



### 1. How do you verify a new cluster really works?

Bottom-up: all nodes Ready; system Pods Running; API server healthy ( `/readyz` ); DNS resolves from inside a Pod; a test Deployment rolls out; its Service is reachable from outside; a Pod on one node reaches a Pod on another; client and server versions are within the supported skew. The order points at the broken layer when a check fails. ([docs/11](#))